DRAFT

v0.1.0

Jacob Repp jrepp@ibm.com

DRAFT

Abstract

This book documents the embedding and retrieval stack built around local `llama-server` inference. It covers the foundations of how embeddings work, how to evaluate and select embedding models, and how to assemble a complete Retrieval-Augmented Generation (RAG) pipeline running on consumer hardware.

The primary target is a local-first deployment: models served via `llama-server` from GGUF artifacts, embeddings exposed through an OpenAI-compatible HTTP endpoint, and a retrieval backend that runs without external infrastructure. The Qwen3 Embedding family is treated as the leading local candidate throughout.

The book is organized into five parts:

1. **Foundations** — what embeddings are, and how tokenization shapes what models can process
2. **Benchmarks and Model Selection** — reading MTEB and RTEB scores, selecting for local RAG
3. **Retrieval Architecture** — dense, sparse, and ColBERT retrieval; ANN indexes; BM25 and RRF
4. **Building the Pipeline** — chunking strategies, the full RAG pipeline, and storage backend comparison
5. **Local Deployment** — serving with llama-server, query instructions, and operational guidance

DRAFT

DRAFT

Table of Contents

Abstract **i**

I Foundations **1**

1 What Is an Embedding? **3**
1.1 The Core Problem . . . 3
1.2 What "Similar Vectors" Means Geometrically . . . 3
1.3 What Training Means . . . 4
1.4 What the Dimensions Represent . . . 4
1.5 From Words to Sentences to Documents . . . 4
1.6 Embedding Models vs Generative Models . . . 5
1.7 Bi-encoder vs Cross-encoder . . . 5
1.8 Why High Dimensionality? . . . 6
1.9 What Makes a Good Embedding Space for Retrieval . . . 6
1.10 Summary . . . 7

2 Tokenization **9**
2.1 Why Tokens, Not Words or Characters? . . . 9
2.2 BPE (Byte Pair Encoding) . . . 9
2.3 WordPiece . . . 10
2.4 SentencePiece . . . 10
2.5 Token Count vs Character Count . . . 10
2.6 How Token Limits Affect Chunking . . . 11
2.7 Tokens and Embedding Quality . . . 12
2.8 Special Tokens . . . 12
2.9 Counting Tokens Without Running the Model . . . 12
2.10 Practical Takeaways . . . 13

II Benchmarks and Model Selection **15**

3 Evaluating Embedding Models: MTEB **17**
3.1 What MTEB Measures . . . 17
3.2 MTEB Task Taxonomy . . . 17

DRAFT

3.3 How to Read an MTEB Score . . . 18
3.4 Common MTEB Metrics . . . 19
3.4.1 What `@k` Means . . . 19
3.4.2 nDCG — Normalized Discounted Cumulative Gain . . . 19
3.4.3 MAP — Mean Average Precision . . . 20
3.4.4 Recall@k . . . 20
3.4.5 Clustering Metrics . . . 20
3.5 Which MTEB Metrics Matter Most for RAG . . . 20
3.6 Qwen3 Series: What It Targets on MTEB . . . 21

4 RTEB and Generalization **23**
4.1 What RTEB Is Trying to Fix . . . 23
4.2 Why RTEB Matters More Than Generic MTEB for Local RAG . . . 23
4.3 The Core Design: Open + Private Datasets . . . 24
4.4 Default Metric: nDCG@10 . . . 24
4.5 What Kinds of Retrieval RTEB Covers . . . 24
4.6 How to Read RTEB for Your Use Case . . . 25
4.7 What RTEB Still Does Not Solve . . . 25
4.8 What RTEB Shows About Qwen3-Embedding . . . 25
4.9 Benchmark-Reading Order for Local RAG . . . 26

5 Embedding Model Selection **27**
5.1 Status . . . 27
5.2 Shortlist . . . 27
5.2.1 Qwen3-Embedding-4B-GGUF . . . 27
5.2.2 Qwen3-Embedding-0.6B-GGUF . . . 28
5.2.3 Qwen3-Embedding-8B-GGUF . . . 28
5.2.4 BAAI/bge-m3 . . . 28
5.2.5 mixedbread-ai/mxbai-embed-large-v1 . . . 29
5.2.6 nomic-embed-text-v1.5-GGUF . . . 29
5.3 Summary Table . . . 29
5.4 Final Recommendation . . . 30
5.5 Recommended Next Steps . . . 30

III Retrieval Architecture **31**

6 Retrieval Architectures **33**
6.1 Dense Retrieval . . . 33
6.2 Sparse Retrieval (SPLADE) . . . 33
6.3 ColBERT — Multi-vector / Late Interaction . . . 33
6.4 Hybrid Retrieval . . . 34
6.5 Retrieval vs Re-ranking: The Two-Stage Pattern . . . 35
6.5.1 Stage 1: Ranking (Retrieval) . . . 35
6.5.2 Stage 2: Re-ranking . . . 35
6.5.3 Why Not Rerank Everything? . . . 35
6.5.4 What Each Stage Optimizes For . . . 35

DRAFT

6.5.5 Practical Retrieval Stack . . . 36
6.6 Architecture Selection Guide . . . 36

7 ANN Indexes and Distance Metrics 37
7.1 The Problem with Exact Search . . . 37
7.2 HNSW — Hierarchical Navigable Small World . . . 37
7.3 Flat Scan (Brute Force) . . . 38
7.4 IVFFlat (Inverted File) . . . 38
7.5 Product Quantization (PQ) . . . 38
7.6 Distance Metrics . . . 38
7.6.1 Cosine Similarity . . . 38
7.6.2 L2 Distance (Euclidean Distance) . . . 39
7.6.3 Dot Product . . . 39
7.7 Elbow Cutoff (Score Cliff Detection) . . . 39

8 Ranking, BM25, and RRF 41
8.1 BM25 — Best Match 25 . . . 41
8.2 RRF — Reciprocal Rank Fusion . . . 41
8.3 nDCG in Practice: Using Rankings to Make Better Decisions . . . 42
8.3.1 Why nDCG Is Better Than Plain Recall for Ranking Quality . . . 42
8.3.2 Why `@10` Is Common . . . 42
8.3.3 nDCG with Binary vs Graded Judgments . . . 42
8.3.4 What nDCG Does Not Tell You . . . 43
8.4 SPO and Entity Graphs . . . 43
8.5 Choosing the Right Retrieval Mix . . . 44

IV Building the Pipeline 45

9 Chunking Strategies 47
9.1 Why Chunk at All? . . . 47
9.2 The Chunk Size Tradeoff . . . 47
9.3 Strategy 1: Fixed-Size Chunking . . . 48
9.4 Strategy 2: Fixed-Size with Overlap . . . 48
9.5 Strategy 3: Sentence-Boundary Chunking . . . 49
9.6 Strategy 4: Recursive Character Splitting . . . 49
9.7 Strategy 5: Structure-Aware Chunking . . . 49
9.8 Strategy 6: Semantic Chunking . . . 50
9.9 Strategy 7: Parent-Child (Small-to-Big) Chunking . . . 50
9.10 Strategy 8: Late Chunking . . . 51
9.11 How memvid Handles Chunking . . . 51
9.12 Practical Checklist . . . 52

DRAFT

10 The RAG Pipeline **53**
10.1 Two Phases . . . 53
10.2 Indexing Phase . . . 53
10.2.1 Step 1: Document Ingestion . . . 53
10.2.2 Step 2: Text Cleaning . . . 54
10.2.3 Step 3: Chunking . . . 54
10.2.4 Step 4: Embedding . . . 54
10.2.5 Step 5: Storing in the Index . . . 55
10.2.6 Indexing Pipeline Summary . . . 55
10.3 Query Phase . . . 55
10.3.1 Step 1: Receive the Query . . . 55
10.3.2 Step 2: Embed the Query . . . 56
10.3.3 Step 3: Retrieve Candidates . . . 56
10.3.4 Step 4: Filter . . . 56
10.3.5 Step 5: Rerank (Optional but Recommended) . . . 56
10.3.6 Step 6: Assemble Context . . . 57
10.3.7 Step 7: Generate the Answer . . . 57
10.4 Common Failure Modes . . . 57
10.4.1 Retrieval Failures . . . 57
10.4.2 Generation Failures . . . 58
10.4.3 Indexing Failures . . . 58
10.5 This Repo's Stack . . . 58

11 Retrieval Backends **61**
11.1 memvid v2 — Deep Analysis . . . 61
11.1.1 What It Is . . . 61
11.1.2 Storage and Embedding Mechanics . . . 61
11.1.3 Retrieval Patterns . . . 62
11.1.4 Memory and Agent Features . . . 62
11.1.5 Performance Profile . . . 62
11.1.6 Hard Limitations . . . 63
11.2 Backend Comparison Matrix . . . 63
11.3 Key Backend Details . . . 63
11.3.1 Qdrant . . . 63
11.3.2 LanceDB . . . 64
11.3.3 MongoDB Atlas . . . 64
11.3.4 pgvector . . . 64
11.4 Decision Guide . . . 64
11.4.1 Stacking memvid with Another Backend . . . 64

V Local Deployment **67**

12 llama-server and llama-embedding **69**
12.1 What llama-embedding Does . . . 69
12.2 What Embedding Generation Means at the Model Level . . . 69
12.3 Why Pooling Matters . . . 69

DRAFT

12.4 llama-embedding vs llama-server `--embedding` . . . 70
12.5 Why Embedding Models Are Often Served Separately . . . 70
12.6 What Gets Indexed . . . 71
12.7 Why Normalization Matters . . . 71
12.8 Why Embedding Performance Differs from Generation Performance . . . 71
12.9 Practical llama-server Invocations . . . 71
12.10 Practical Deployment Pattern for This Repo . . . 72

13 Query Instructions and Operational Guidance **73**
13.1 Why Query-Side Instructions Help . . . 73
13.2 The Basic Pattern . . . 73
13.3 Why Not Instruct Both Sides . . . 74
13.4 What "Customize the Instruct" Really Means . . . 74
13.5 Example Instruction Templates . . . 75
13.6 What the Reported 1%–5% Drop Means . . . 75
13.7 Operational Implication . . . 75
13.8 Operational Checklist . . . 76
13.8.1 Before First Deployment . . . 76
13.8.2 After Deployment . . . 76
13.8.3 Upgrading the Stack . . . 76

Glossary **77**

DRAFT

DRAFT

Part I

Foundations

DRAFT

DRAFT

1

What Is an Embedding?

This is the foundational question for everything in this book. Retrieval, reranking, memory, hybrid search, and ANN indexes are all downstream of the answer.

1.1 The Core Problem

Computers are good at numbers. Language is not numbers. There is a gap.

The naive approach to bridging that gap is to assign each word an integer ID:

```
"cat"    → 4821
"dog"    → 9302
"feline" → 2201
```

This does not work for meaning-sensitive tasks. The number 4821 is not close to 2201, but "cat" and "feline" mean nearly the same thing. The integer representation destroys all information about semantic relationships.

An embedding is a different kind of representation: instead of one integer, each word or text gets a **vector**—a list of floating-point numbers:

```
"cat"    → [0.21, -0.45, 0.03, 0.88, ...]  (1536 numbers)
"feline" → [0.23, -0.42, 0.01, 0.91, ...]  (1536 numbers)
"dog"    → [0.18,  0.31, 0.55, 0.12, ...]  (1536 numbers)
```

The key property: **similar meanings produce similar vectors**. "cat" and "feline" are close in the vector space. "cat" and "carburetor" are far apart.

1.2 What "Similar Vectors" Means Geometrically

Think of each vector as a point in a high-dimensional space. With 1536 dimensions you cannot visualize it, but the geometry still works the same way as 2D or 3D:

- Two points can be **close** (nearby in space)
- Two points can be **far apart** (distant in space)
- A direction in space can represent a concept

DRAFT

The classic illustration uses word vectors trained on large text corpora:

```
vector("king") - vector("man") + vector("woman")   vector("queen")
```

The direction from "man" to "woman" captures the concept of gender. The direction from "king" to "queen" goes the same way. The arithmetic works because the geometry of the space reflects the structure of language. This is not programmed in—it emerges from training.

1.3 What Training Means

An embedding model is trained on a large corpus of text. The training process adjusts the model's weights so that texts that should be similar end up nearby in vector space.

The most common training objective for modern embedding models is **contrastive learning**:

```
Given: (query, relevant_document, irrelevant_document)
Goal:  make vector(query) close to vector(relevant_document)
       make vector(query) far from vector(irrelevant_document)
```

This is called a **triplet** or **(anchor, positive, negative)** setup. The loss function penalizes the model when it places a query closer to an irrelevant document than to a relevant one.

The irrelevant documents matter a lot. Easy negatives (completely unrelated texts) produce weak models. **Hard negatives**—documents that look relevant but are not—force the model to learn fine-grained distinctions. Most state-of-the-art embedding models are trained with carefully mined hard negatives.

After training on millions or billions of such triplets, the model has learned a vector space where semantic similarity corresponds to geometric proximity.

1.4 What the Dimensions Represent

This is a common point of confusion: no individual dimension means anything specific.

You cannot point to dimension 742 and say "this is the adjective-ness dimension." The model learns a distributed representation where meaning is encoded across all dimensions simultaneously. This is fundamentally different from a hand-crafted feature vector.

What the dimensions collectively encode is a **projection of meaning into a space optimized for the similarity task the model was trained on**. The training corpus and training objective together determine what the space looks like.

Practical consequence: embeddings from different models are **not comparable**. A vector from `text-embedding-3-small` and a vector from `nomic-embed-text-v1.5` live in completely different spaces. You cannot mix them in the same index.

1.5 From Words to Sentences to Documents

Early word embedding models (Word2Vec, GloVe) produced one vector per word. The same word always got the same vector, regardless of context.

Modern embedding models (BERT, RoBERTa, and their successors) are **contextual**. The same word gets a different vector depending on the surrounding text:

DRAFT

```
"bank" in "I went to the bank to deposit money"
    → vector reflecting financial institution

"bank" in "we sat on the bank of the river"
    → vector reflecting a riverbank
```

These models produce one vector per **token** (a subword unit—see Chapter 2), and then **pool** those token vectors into a single vector representing the whole input.

Common pooling strategies:

Strategy	What it does	Typical use
Mean pooling	Average all token vectors	Most embedding models
CLS pooling	Use the special `[CLS]` token vector	BERT-style models
Last-token pooling	Use the final token's vector	Decoder models (Qwen3)

The pooling strategy is baked into how the model was trained. Using the wrong pooling for a given model degrades quality significantly.

1.6 Embedding Models vs Generative Models

Both types of model can run on `llama-server`, but they serve different purposes:

	Embedding model	Generative model
Input	Text	Text (plus history, tools)
Output	A vector of numbers	More text
Purpose	Represent meaning for retrieval	Produce responses
Runs over input	Once (encode)	Autoregressively
Typical size	0.1B–8B params	1B–400B+ params
Inference cost	One forward pass	Many forward passes

An embedding model reads the input once and produces a fixed-size vector. It never generates text. A generative model reads the input and then generates text one token at a time, with each token depending on all previous tokens.

For RAG, both are needed: the embedding model retrieves relevant context; the generative model uses that context to produce an answer.

1.7 Bi-encoder vs Cross-encoder

Both are transformer models, but they operate differently.

Bi-encoder (embedding model):

DRAFT

```
query  → [encoder] → query_vector
doc    → [encoder] → doc_vector
score  = similarity(query_vector, doc_vector)
```

The query and document are encoded independently. This is fast because document vectors can be precomputed and stored in an index.

Cross-encoder (reranker):

```
[query + doc] → [encoder] → relevance_score
```

The query and document are read together. The model can reason about their interaction directly. This is slower because it cannot be precomputed—you must run the model fresh for every (query, candidate) pair at query time.

Bi-encoders retrieve at scale. Cross-encoders rerank at precision. The typical production stack uses both in sequence: bi-encoder fetches top 50, cross-encoder reranks to top 5.

1.8 Why High Dimensionality?

Embedding dimensions range from 384 (small, fast models) to 3072 (large, high-quality models). More dimensions means:

- More capacity to encode nuanced distinctions
- Larger index size (1536-dim float32 = 6 KB per vector)
- Slower distance computation (mitigated by ANN algorithms)

The relationship is not linear. Going from 384 to 768 dims captures meaningfully more information. Going from 1536 to 3072 dims produces smaller gains for most retrieval tasks.

Dimensionality reduction is possible: `text-embedding-3-small` and `text-embedding-3-large` support a `dimensions` parameter that truncates the output while preserving most signal. This is useful when index storage is the bottleneck.

1.9 What Makes a Good Embedding Space for Retrieval

The embedding space has to work for the query-document relationship, not just for document-document similarity. A space that groups similar documents together is not automatically good at matching short queries to long documents.

Key properties for retrieval:

- **Asymmetric matching**: a short query ("python list sorting") should be close to a long document explaining `list.sort()`, even though they differ greatly in length and style
- **Out-of-vocabulary robustness**: the model should handle domain-specific terms it has not seen exactly during training
- **Cross-lingual capability** (when needed): a query in English and a document in Chinese should still match if they express the same concept

This is why embedding benchmarks (MTEB, RTEB) test retrieval specifically, not just semantic textual similarity. See Chapter 3 and Chapter 4.

DRAFT

1.10 Summary

- An embedding turns text into a vector of numbers
- Similar meanings produce similar vectors; the geometry reflects semantics
- This emerges from contrastive training on (anchor, positive, negative) triplets
- Individual dimensions mean nothing; meaning is distributed across all of them
- Embeddings from different models are incompatible — never mix them in one index
- Pooling strategy (mean, CLS, last-token) is part of the model contract
- Bi-encoders are fast (precomputed vectors); cross-encoders are precise (joint reasoning)
- More dimensions = more capacity, larger storage, not always worth the cost

DRAFT

DRAFT

2

Tokenization

Tokenization is the process of splitting text into the discrete units a model processes. Understanding it is essential for working with embedding models because token counts—not character counts—determine whether a text fits in a model's context window, how chunking interacts with model limits, and why costs and latency vary with input length.

2.1 Why Tokens, Not Words or Characters?

Early NLP systems split on whitespace: each word is a unit. This breaks down on several practical problems:

- **Vocabulary explosion.** English has hundreds of thousands of words, and new ones are coined constantly. A word-level vocabulary must either be huge or mark unknown words as `[UNK]`, losing all information about them.
- **Morphological variants.** "run", "runs", "running", and "ran" are four different vocabulary entries that share almost all meaning. A word-level model treats them as unrelated.
- **Multilinguality.** Japanese and Chinese have no spaces. Arabic and Hebrew write without vowels. A word tokenizer that splits on spaces fails silently.

Character-level tokenization solves the vocabulary problem (26 letters plus punctuation is a tiny vocabulary) but produces very long sequences. Attention is $O(n^2)$ in sequence length, so character sequences are expensive to process.

Subword tokenization—the dominant approach—sits between the two. Common words stay as single tokens. Rare words are split into meaningful subword pieces. Sequences stay short enough for efficient attention.

2.2 BPE (Byte Pair Encoding)

BPE is the most widely used tokenization algorithm (GPT-2 through GPT-4o, LLaMA, Qwen, Mistral).

How it works:

1. Start with a vocabulary of individual characters (or bytes).

DRAFT

2. Count all adjacent pairs in the training corpus.
3. Merge the most frequent pair into a new token.
4. Repeat until the vocabulary reaches the target size (typically 32k–128k tokens).

Example:

```
Training text: "low lower lowest lower low"

Initial: l o w  l o w e r  l o w e s t  l o w e r  l o w
Step 1: merge 'l'+'o' → 'lo' (most frequent pair)
        lo w  lo w e r  lo w e s t  lo w e r  lo w
Step 2: merge 'lo'+'w' → 'low'
        low  low e r  low e s t  low e r  low
Step 3: merge 'low'+'e' → 'lowe'
        low  lowe r  lowe s t  lowe r  low
```

After training on a large corpus, common words like "the", "is", and "model" are single tokens. Rare words split into recognizable subwords: "tokenization" becomes `["token", "ization"]`. Truly novel words split further: `"Qwen3Embedding"` → `["Qw", "en", "3", "Embed", "ding"]`.

The **byte-level variant** (used by GPT-2 and descendants) operates on raw bytes rather than characters. This ensures any Unicode text can be tokenized without an unknown-token fallback, at the cost of longer sequences for non-Latin scripts.

2.3 WordPiece

Used by BERT and its descendants (including many embedding models like BGE and nomic-embed-text). Conceptually similar to BPE, but merges are chosen to maximize the likelihood of the training data under a language model, not just by raw pair frequency. Subword pieces are prefixed with `##` to indicate continuation:

```
"playing"     → ["play", "##ing"]
"unplayable"  → ["un", "##play", "##able"]
```

2.4 SentencePiece

Used by LLaMA, Mistral, and many multilingual models. Operates directly on raw text without pre-tokenization—no whitespace splitting required first. Handles languages without spaces naturally. The underlying algorithm is typically BPE or Unigram.

2.5 Token Count vs Character Count

A critical practical point: **model limits are in tokens, not characters or words**.

Approximate conversions for English text:

DRAFT

Unit	Typical ratio
1 token	~4 characters
1 token	~0.75 words
100 tokens	~75 words, ~400 characters
1,000 tokens	~750 words, ~3–4 paragraphs

These ratios are for English. Other languages differ:

Language	Tokens per word (approx)
English	1.3
Spanish / French	1.4
German	1.5 (compound words split more)
Chinese	1.5–2.5 (ideographs often multi-byte)
Arabic	1.5–2.0
Code (Python)	2–4 (identifiers, indentation, symbols)

Code tokenizes much less efficiently than prose. A 100-line Python function may consume 300–500 tokens. This matters for code retrieval use cases where chunk size must be set by token count, not character count.

2.6 How Token Limits Affect Chunking

Every embedding model has a maximum context length:

Model	Max tokens
BGE-small-en-v1.5	512
nomic-embed-text-v1.5	8,192
text-embedding-3-small	8,191
text-embedding-3-large	8,191
Qwen3-Embedding series	32,768

What happens when input exceeds the limit depends on the implementation:

- **Truncation:** the model silently discards tokens beyond the limit. The embedding represents only the first N tokens. Content in the truncated portion is invisible to retrieval.
- **Error:** the API or model raises an error. Callers must pre-split.

Truncation is the silent failure mode. If your chunks regularly exceed the model's token limit, you are losing content without any warning.

DRAFT

Chunk size should be set in tokens when precision matters. For a 512-token limit model with a 15% safety margin, target 435-token chunks. Converted to characters at 4 chars/token: roughly 1,740 characters. memvid's default of 1,200 characters is conservative for 512-token models and well within limits for 8K+ token models.

2.7 Tokens and Embedding Quality

Shorter inputs produce different embeddings than longer ones because pooling collapses all token vectors into one. A 5-token chunk and a 500-token chunk are both represented by a single 1536-dim vector, but the 500-token vector averages over far more content.

Implications:

- **Very short chunks** (1–3 sentences) embed well—the vector captures the specific topic of that chunk cleanly.
- **Very long chunks** produce embeddings that represent a blend of all topics in the chunk. They retrieve for any of those topics at reduced precision.
- **Asymmetric retrieval:** a 10-word query embedding will not be close to a 2,000-token document embedding even if the document contains the answer, because the document's vector is averaged over content unrelated to the query. This is why chunking exists.

2.8 Special Tokens

Most models use reserved tokens for structural purposes:

Token	Meaning	Models
`[CLS]`	Classification / sentence start	BERT-style
`[SEP]`	Separator between segments	BERT-style
`[PAD]`	Padding to uniform length	All
`[UNK]`	Unknown token (rare in modern BPE)	All
`<s>`	Sequence start	LLaMA/Mistral
`</s>`	Sequence end	LLaMA/Mistral
`<|endoftext|>`	End of text	GPT-2/3/4

Special tokens consume context window budget. A BGE model with a 512-token limit uses 2 special tokens (`[CLS]` and `[SEP]`), leaving 510 tokens for content.

For CLS pooling models, the `[CLS]` token is the output used as the embedding. For last-token pooling models (Qwen3), the final real token before `</s>` is used.

2.9 Counting Tokens Without Running the Model

Counting tokens before sending text to a model is useful for ensuring chunks stay within limits, estimating API cost, and debugging truncation issues.

DRAFT

Python (HuggingFace tokenizers):

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("BAAI/bge-small-en-v1.5")
tokens = tokenizer.encode("The quick brown fox")
print(len(tokens))  # 6
```

Python (tiktoken, for OpenAI models):

```
import tiktoken

enc = tiktoken.get_encoding("cl100k_base")  # text-embedding-3-*
tokens = enc.encode("The quick brown fox")
print(len(tokens))  # 5
```

Rule of thumb without a tokenizer: Divide character count by 4 for English prose. Accurate to ±20% for most inputs. For code, divide by 3. For non-Latin scripts, divide by 2.

2.10 Practical Takeaways

- Token limits are per-model and in tokens, not characters. Check the model card.
- English prose: ~4 characters per token. Code: ~3 chars per token. CJK: ~2 chars per token.
- Exceeding the limit silently truncates in most implementations.
- Set chunk size in tokens when working with models near their 512-token limit; character estimates are close enough for 8K+ token models.
- Short chunks embed their specific topic precisely; long chunks produce averaged embeddings.
- Special tokens consume 2–4 tokens of context budget per call.

DRAFT

DRAFT

Part II

Benchmarks and Model Selection

DRAFT

DRAFT

3

Evaluating Embedding Models: MTEB

MTEB stands for **Massive Text Embedding Benchmark**. It is the primary public benchmark suite for evaluating embedding models across many task families rather than a single retrieval dataset. That breadth is why it is useful—and also why it is easy to overread.

3.1 What MTEB Measures

MTEB evaluates models across a mix of task categories:

- Retrieval
- Reranking
- Classification
- Clustering
- Semantic Textual Similarity (STS)
- Pair classification
- Summarization-related matching
- Bitext mining

The exact benchmark composition depends on which leaderboard slice you are looking at: English MTEB, multilingual MTEB (MMTEB), or code-focused subsets such as MTEB-Code. A single leaderboard score is a compressed summary across many task types, domains, and datasets.

3.2 MTEB Task Taxonomy

The official MTEB task taxonomy includes these core text task families:

`Retrieval` Retrieve relevant documents for a query from a corpus.

DRAFT

`Reranking` Reorder a candidate list for better top-of-list relevance.

`Classification` Assign a label to a text.

`MultilabelClassification` Assign multiple labels to a text.

`Clustering` Group texts by semantic similarity without labels.

`PairClassification` Classify the relationship between two texts.

`STS` Semantic textual similarity between two texts.

`Summarization` Evaluate summary-document semantic correspondence.

`BitextMining` Find semantically matching sentence pairs across languages.

`InstructionRetrieval` Retrieval tasks where instruction prompting is part of evaluation.

`InstructionReranking` Reranking tasks where instruction prompting is part of evaluation.

For a local RAG stack, these are the most important by priority:

1. `Retrieval`
2. `Reranking`
3. `InstructionRetrieval`
4. `InstructionReranking`
5. `BitextMining` (if multilingual search matters)

Everything else is useful background signal, but it is not the center of gravity for local RAG.

3.3 How to Read an MTEB Score

A high MTEB score generally means the model is broadly capable across many embedding tasks. It does not automatically mean:

- It is best for your exact workload
- It is fastest
- It is the easiest to serve locally
- It is best for first-stage retrieval versus reranking

For local RAG work, MTEB should be treated as a strong screening benchmark—not the only decision criterion.

DRAFT

3.4 Common MTEB Metrics

MTEB does not use one single metric for every task. It uses task-appropriate metrics, then aggregates results.

3.4.1 What `@k` Means

When you see metrics like `nDCG@10`, `Recall@20`, or `Precision@5`, the `@k` part means: evaluate only the top k ranked items.

- `nDCG@10` — score the ranking quality of only the top 10 retrieved items
- `Recall@20` — how many relevant items were captured inside the top 20
- `pass@1` — did the first attempt succeed?

Real systems almost never use the entire ranked list. They typically use top 5, top 10, or top 20. That is why metrics with `@k` are often more meaningful than a metric over an unbounded ranking.

3.4.2 nDCG — Normalized Discounted Cumulative Gain

nDCG is a ranking metric. Intuitively:

- It rewards returning relevant items near the top
- It rewards putting the most relevant items earlier in the list
- It penalizes good results buried lower in the ranking

nDCG is built from two pieces: DCG (Discounted Cumulative Gain) and IDCG (Ideal DCG). The normalized score is:

$$\text{nDCG@k} = \frac{\text{DCG@k}}{\text{IDCG@k}}$$

The DCG formula with graded relevance is:

$$\text{DCG@k} = \sum_{i=1}^{k} \frac{2^{\text{rel}_i} - 1}{\log_2(i+1)}$$

Worked example. Suppose the top-5 retrieved documents have relevance labels $A = [3, 2, 0, 1, 0]$:

$$\begin{aligned}\text{DCG}_A\text{@5} &= \frac{7}{1} + \frac{3}{1.585} + \frac{0}{2} + \frac{1}{2.322} + \frac{0}{2.585} \\ &\approx 7 + 1.893 + 0 + 0.431 + 0 = 9.324\end{aligned}$$

The ideal ordering $[3, 2, 1, 0, 0]$ gives IDCG@5 ≈ 9.393.

DRAFT

$$\text{nDCG}_A@5 = \frac{9.324}{9.393} \approx 0.993 \quad \text{(near-ideal ranking)}$$

A worse ordering $B = [0, 1, 2, 3, 0]$ of the same documents gives $\text{nDCG}_B@5 \approx 0.548$—significantly lower, even though both rankings retrieved the same relevant items. nDCG cares about **order**, not just membership.

3.4.3 MAP — Mean Average Precision

MAP measures how well relevant items are retrieved across the ranking. It is sensitive to whether relevant results appear early, not just whether they appear somewhere. Often used when there may be multiple relevant documents per query.

3.4.4 Recall@k

Recall@k asks: among all relevant items, how many were captured in the top k results? This is a coverage metric. It is less concerned with the exact order inside the top k than nDCG. This matters when your pipeline does first-stage retrieval with a generous top-k followed by second-stage reranking: high recall at the retrieval stage is often more important than perfect ranking at that stage.

3.4.5 Clustering Metrics

Clustering tasks often use metrics such as V-measure or NMI (Normalized Mutual Information). If the embedding space is good, unsupervised clusters should align better with real semantic categories.

3.5 Which MTEB Metrics Matter Most for RAG

If your practical use case is local RAG, the most important categories are:

- Retrieval metrics like nDCG and Recall@k
- Reranking metrics if you plan to add a reranker
- Multilingual retrieval metrics if your corpus or queries span multiple languages

For a two-stage retrieval pipeline:

- Stage-1 embedding retriever should have strong recall
- Stage-2 reranker should improve top-of-list ordering

Practical rule for local retrieval: Use `Recall@k` to judge whether your first-stage embedding retriever is finding enough candidates. Use `nDCG@k` to judge whether the highest-ranked candidates are in the right order. Use reranking to improve `nDCG@k` after retrieval if recall is already good.

DRAFT

3.6 Qwen3 Series: What It Targets on MTEB

The Qwen3 Embedding and Qwen3 Reranker series are designed as a matched retrieval stack rather than isolated single-purpose models. The official model cards describe the series as targeting:

- Text retrieval
- Code retrieval
- Text classification
- Text clustering
- Bitext mining

These tasks are related, but not identical. For a local RAG use case, text retrieval and code retrieval are the most relevant. The series reports strong MTEB scores: Qwen3-Embedding-4B at 69.45 on multilingual MTEB and 74.60 on MTEB English v2 as of June 2025.

MTEB Score Interpretation: High MTEB scores may reflect benchmark proximity rather than out-of-distribution generalization. Models trained on data that overlaps with MTEB test sets can achieve inflated public scores. See Chapter 4 for the RTEB benchmark designed to detect this.

DRAFT

DRAFT

4

RTEB and Generalization

DRAFT

If retrieval is your main use case, RTEB is more relevant than the overall MTEB aggregate. RTEB stands for **Retrieval Text Embedding Benchmark**. The Hugging Face launch post was published on October 1, 2025, by the MTEB team.

4.1 What RTEB Is Trying to Fix

RTEB was introduced because broad public retrieval leaderboards have a known problem: models get repeatedly tested against the same public datasets, and training data can overlap with benchmark data. Leaderboard gains can partly reflect benchmark familiarity rather than true retrieval generalization.

The RTEB authors frame this as a generalization gap between public benchmark performance and performance on new, unseen retrieval problems. The benchmark is explicitly trying to be:

- Retrieval-first
- More realistic
- More resistant to benchmark overfitting

The Hugging Face post makes a useful distinction between a *leaderboard score* and a *zero-shot score*. A model can look strong on a familiar public benchmark while still generalizing poorly to new retrieval tasks.

4.2 Why RTEB Matters More Than Generic MTEB for Local RAG

Your actual local use case is not broad embedding capability across classification, clustering, STS, and bitext mining. It is: can this model retrieve the right chunks for my queries?

That makes RTEB a better decision aid because it focuses on retrieval quality, domain realism, and multilingual and specialized search settings.

4.3 The Core Design: Open + Private Datasets

RTEB uses a hybrid benchmark design:

- **Open datasets** — fully public and reproducible
- **Private datasets** — evaluated by the maintainers to test generalization to unseen data

This is the most important thing to understand about RTEB. Strong public-only performance can be misleading. If a model drops sharply on the private side, that suggests benchmark overfitting or weak generalization.

For transparency, the maintainers provide descriptive statistics, dataset descriptions, and sample (query, document, relevance) triplets. The benchmark balances reproducibility with anti-overfitting protection.

4.4 Default Metric: nDCG@10

The RTEB material explicitly states that the default leaderboard metric is `nDCG@10`. That is a strong choice for retrieval because it emphasizes whether relevant documents appear near the top—which is exactly what matters when a RAG pipeline passes only the top 5 or top 10 chunks into the final prompt.

4.5 What Kinds of Retrieval RTEB Covers

RTEB is not one dataset. It is a retrieval benchmark family spanning multiple domains and languages. The benchmark currently covers 20 languages, with datasets organized into groups by domain and language. One dataset can belong to multiple groups—a German legal dataset counts as both `german` and `legal`.

Domain coverage includes:

- **Legal:** AILACasedocs, AILAStatutes, LegalSummarization, LegalQuAD
- **Finance:** FinanceBench, HC3Finance, FinQA
- **Code:** HumanEval, MBPP, APPS, DS1000, WikiSQL
- **Healthcare:** ChatDoctor_HealthCareMagic, HC3 Medicine, Cure, TripClick
- **Multilingual and general:** MIRACLHardNegatives, JaQuAD, FreshStack

RTEB benchmark names in the catalog include:

- `HumanEvalRetrieval`, `MBPPRetrieval`, `WikiSQLRetrieval`
- `FreshStackRetrieval`, `FinanceBenchRetrieval`
- `ChatDoctorRetrieval`, `MIRACLRetrievalHardNegatives`

This is important because retrieval quality is highly domain-sensitive. A model that looks good on generic QA-style retrieval may not be the best model for code search, technical docs, multilingual knowledge bases, or legal corpora.

DRAFT

4.6 How to Read RTEB for Your Use Case

You should not ask: "which model is #1 on RTEB overall?"

You should ask: "which model performs well on the RTEB slice closest to my workload?"

- If your corpus is mostly English docs, `RTEB(eng, beta)` matters more than the multilingual aggregate
- If your main problem is code retrieval, the code-heavy tasks matter more than finance or legal tasks
- If your content spans multiple languages, the multilingual and language-specific slices matter more than English-only results

RTEB slices relevant to local infrastructure workloads:

`RTEB(beta)` Broad retrieval signal across domains

`RTEB(eng, beta)` Best general-purpose check if most material is English

Code-heavy tasks For source code, documentation, or engineering notes

Multilingual retrieval tasks If knowledge base or queries are not strictly English

Language-specific subsets include `RTEB(deu, beta)`, `RTEB(fra, beta)`, and `RTEB(jpn, beta)`.

4.7 What RTEB Still Does Not Solve

RTEB is better aligned with retrieval than the broad MTEB aggregate, but it is not perfect. The official launch notes several current limitations:

- It is still in beta
- It is text-only
- Language coverage is still expanding
- About half of the current retrieval datasets are repurposed from QA datasets

QA-derived retrieval can favor lexical overlap more than true production retrieval sometimes does. So even with RTEB, your own internal retrieval eval still matters.

4.8 What RTEB Shows About Qwen3-Embedding

Qwen3-Embedding does not have published first-party RTEB scores as of this writing. That absence is the core open question in the original model recommendation: MTEB rank-1 without a corresponding RTEB result means the generalization question is unanswered, not confirmed.

Indirect evidence:

DRAFT

- Octen-Embedding-8B, fine-tuned on Qwen3-8B base, scores **0.8045 on RTEB overall** and performs slightly higher on private data than public (0.8157 private vs 0.7953 public). That inverted gap is a strong generalization signal and suggests the Qwen3 foundation model is not contaminated.
- Qwen3-Embedding's three-stage training (contrastive pretraining on weakly supervised data → supervised fine-tuning → model merging) is designed for domain breadth, which is consistent with generalization.
- Qwen3-Embedding outperforms BGE-M3 by ~18% on MTEB English v2 and ~8% on MMTEB, a larger margin than benchmark contamination alone would typically produce.

Tentative interpretation: Qwen3-Embedding is probably not severely contaminated, but "probably not" is not the same as confirmed.

RTEB-confirmed alternatives:

Model	RTEB score	Notes
Octen-Embedding-8B	0.8045	Fine-tuned on Qwen3-8B; best published RTEB result
Voyage-3-large	0.7812	API-only; no local serving
Gemini-embedding-001	0.7602	API-only
BGE-M3	not published	Strong multilingual; GGUF available

For local llama.cpp serving specifically, BGE-M3 is the strongest RTEB-adjacent alternative: it supports dense, sparse (SPLADE), and ColBERT-style retrieval in a single model, has a well-established generalization reputation predating the MTEB contamination era, and 8,192 token context.

4.9 Benchmark-Reading Order for Local RAG

1. Domain-relevant RTEB retrieval results
2. Broader retrieval metrics like MTEB retrieval subsets
3. Overall MTEB aggregate
4. Secondary task families like clustering or classification (background signal only)

Before finalising a model choice: Check whether Qwen3-Embedding RTEB scores have been published since this was written (the RTEB leaderboard is live and updated). Run a small domain-specific internal eval against your actual retrieval corpus — even 50–100 query-document pairs with manual relevance labels will reveal gross distribution mismatch.

DRAFT

5

Embedding Model Selection

DRAFT

This chapter covers model selection for the specific context of this repo:

- Local inference centered on `llama.cpp` / `llama-server`
- The main compatibility target is an OpenAI-compatible local endpoint
- Open WebUI and similar clients are easiest to support when the embedding model is served as a dedicated local embeddings endpoint

5.1 Status

Under Re-evaluation: The original recommendation (Qwen3-Embedding-4B) was based on MTEB scores. After reviewing RTEB research (see Chapter 4), the question of whether Qwen3-Embedding's rankings reflect genuine out-of-distribution generalization or benchmark proximity remains open. The final recommendation below reflects the current best assessment given available evidence.

5.2 Shortlist

5.2.1 Qwen3-Embedding-4B-GGUF

Key findings from the model card:

- 4B parameters, 32k context length
- Embedding dimension up to 2560, configurable output from 32 to 2560 dims
- 100+ language support
- Official GGUF quantizations including Q4_K_M
- Explicit `llama.cpp` usage documented on the model card:

```
llama-embedding ... --pooling last
llama-server -m model.gguf --embedding --pooling last
```

- 69.45 on multilingual MTEB, 74.60 on MTEB English v2

Practical footprint:

- Q4_K_M: ~2.5 GB
- F16: ~8.05 GB

Fit assessment: Best balance of quality, local footprint, and implementation simplicity. Easiest path to adopt because it matches the existing llama.cpp serving model.

5.2.2 Qwen3-Embedding-0.6B-GGUF

Key findings:

- 0.6B parameters, 32k context length, embedding dimension up to 1024
- 100+ language support, instruction awareness, configurable output dimension
- Q8_0: ~639 MB, F16: ~1.2 GB

Fit assessment: Strongest low-footprint candidate. Best choice when memory or startup speed matters more than absolute retrieval quality.

5.2.3 Qwen3-Embedding-8B-GGUF

Key findings:

- 8B parameters, official GGUF release
- Q4_K_M: ~4.68 GB
- Part of the same Qwen3 embedding family with multilingual, long-context, and code retrieval emphasis

Fit assessment: Strongest quality option in the reviewed Qwen family. Probably overkill for a first local deployment unless retrieval quality is the top priority and the extra memory cost is acceptable.

5.2.4 BAAI/bge-m3

Key findings:

- Supports dense, sparse, and multi-vector (ColBERT) retrieval in one model
- 100+ languages, 8,192 token context
- The card explicitly recommends hybrid retrieval plus reranking for RAG

Fit assessment: Technically very capable and a strong retrieval baseline. Less natural fit for this repo than Qwen3 GGUF because the Hugging Face card is centered on the broader BGE ecosystem rather than a simple official GGUF + llama-server serving path.

This model is strongest when you are prepared to build a richer retrieval stack around dense + sparse + reranking, not when you want the cleanest single-model local embeddings server.

DRAFT

5.2.5 mixedbread-ai/mxbai-embed-large-v1

Key findings:

- Strong English embedding model
- Retrieval queries should use the prompt: *"Represent this sentence for searching relevant passages:"*
- Supports Matryoshka representation learning and quantization
- Strong March 2024 MTEB results; compares favorably with `text-embedding-3-large`

Fit assessment: A serious option for English-heavy retrieval, but adopting it cleanly here would likely mean introducing a separate embeddings-serving stack (Infinity, sentence-transformers) instead of reusing the local GGUF tooling pattern.

5.2.6 nomic-embed-text-v1.5-GGUF

Key findings:

- Official GGUF release, explicitly llama.cpp-compatible
- Requires task instruction prefixes such as `search_query:`
- Noted context-extension requirement for the full 8,192-token behavior
- Q4_K_M: ~81 MiB

Fit assessment: Very attractive if tiny footprint is the main requirement. Weaker choice than Qwen3 for a new deployment because it is older and its required task prefixes add operational sharp edges.

5.3 Summary Table

Model	Strength	Weakness	Fit
Qwen3-Embedding-4B	Quality + practicality	Heavier than tiny models	Best overall
Qwen3-Embedding-0.6B	Lightweight	Lower ceiling than 4B/8B	Best lightweight
Qwen3-Embedding-8B	Highest quality	More memory	Best max-quality
BAAI/bge-m3	Dense+sparse versatility	Less direct fit	Good advanced
mxbai-embed-large-v1	Strong English retrieval	Not GGUF-first	Separate stack
nomic-embed-text-v1.5	Extremely small	Older, prefix-sensitive	Tiny fallback

DRAFT

5.4 Final Recommendation

If the goal is "state of the art and compatible for local use":

1. Adopt `Qwen/Qwen3-Embedding-4B-GGUF`
2. Serve it separately from the chat model
3. Expose it as the local embeddings endpoint for RAG and Open WebUI

Why this recommendation is defensible:

- Qwen's Hugging Face cards present the strongest current benchmark story among reviewed candidates
- The family has an official GGUF distribution
- The card explicitly documents `llama.cpp` / `llama-server` usage
- The 4B size is large enough to be serious and small enough to be practical
- The Qwen3-Embedding family pairs with Qwen3-Reranker models for a complete two-stage retrieval stack

Reranker Pairing: Good retrieval systems often use both an embedding model for fast first-pass candidate retrieval and a reranker for slower but more accurate second-pass ordering. The Qwen3-Reranker family maps directly to the embedding family:

- `Qwen/Qwen3-Reranker-0.6B` — lightweight
- `Qwen/Qwen3-Reranker-4B` — balanced
- `Qwen/Qwen3-Reranker-8B` — maximum quality

5.5 Recommended Next Steps

1. Check whether Qwen3-Embedding RTEB scores have been published since this was written (the RTEB leaderboard is live and updated).
2. Run a small domain-specific internal eval against your actual retrieval corpus. Even 50–100 query-document pairs with manual relevance labels will reveal gross distribution mismatch.
3. If internal eval is not feasible, treat Qwen3-Embedding as the leading candidate but defer to RTEB-confirmed alternatives if they appear.
4. If deploying only one model first, the embedding model is the right first step because it gives the core `/v1/embeddings` capability and is the prerequisite for scalable corpus search.

DRAFT

Part III

Retrieval Architecture

DRAFT

DRAFT

6

Retrieval Architectures

Embedding models can produce different kinds of representations. The architecture determines what kind of index you build and how query-document similarity is computed.

6.1 Dense Retrieval

The standard approach. One vector per text.

```
query  → [single vector]
doc    → [single vector]
score  = dot_product(query_vec, doc_vec)
```

Fast at query time. Index is compact. Works well when queries and documents are semantically similar in the embedding space.

Weakness: a single vector must compress all meaning. Queries that need to match multiple distinct aspects of a document can lose signal.

6.2 Sparse Retrieval (SPLADE)

Instead of a dense vector, the model outputs a sparse weighted term vocabulary—like a learned BM25.

```
query → {term: weight, term: weight, ...}  (most weights  0)
doc   → {term: weight, term: weight, ...}
score = sum of overlapping weighted terms
```

Good at exact and near-exact term matching. Complements dense retrieval on queries where lexical specificity matters: version numbers, proper nouns, code tokens.

BGE-M3 includes a sparse retrieval head alongside its dense head, making hybrid pipelines cheaper to operate.

6.3 ColBERT — Multi-vector / Late Interaction

ColBERT keeps one vector per token rather than collapsing to a single vector.

DRAFT

```
query: [what]  [is]  [the]  [capital]
          ↓      ↓     ↓       ↓
          q1     q2    q3      q4

doc:  [Paris] [is] [a] [city] [in] [France]
          ↓      ↓   ↓    ↓      ↓     ↓
          d1     d2  d3   d4     d5    d6
```

Similarity (MaxSim):

```
score = sum over each qi of: max(dot(qi, dj) for all dj)
```

Each query token independently finds its best matching document token, then scores are summed.

Why it matters:

- "bank of a river" and "bank loan" map to the same dense vector in ambiguous models; ColBERT keeps them distinct because *river* and *loan* match different document tokens
- Multi-aspect queries ("fast AND cheap AND local") get each aspect separately matched rather than compressed into one average direction
- Exact token sensitivity: "Python 3.11 migration" won't match a document about "Python 2.7" because *3.11* and *2.7* pull in different directions

Tradeoffs:

	Dense	**Sparse**	**ColBERT**
Index size	small	medium	large (tokens × docs)
Query speed	very fast	fast	slower
Recall on complex queries	good	good for lexical	better
Typical use	first-stage retrieval	hybrid retrieval	reranking or dedicated index

6.4 Hybrid Retrieval

Most production systems combine modes rather than choosing one:

1. Dense retrieval → top-k candidates (fast, high recall)
2. Sparse/ColBERT reranking → reorder top-k (slower, higher precision)

BGE-M3 is notable because it runs all three modes (dense, sparse, ColBERT) from a single set of weights, making hybrid pipelines cheaper to operate than maintaining separate models.

The typical local deployment for this repo uses dense retrieval for first-stage candidate generation and reserves sparse or ColBERT for a second-stage reranking pass.

DRAFT

6.5 Retrieval vs Re-ranking: The Two-Stage Pattern

This distinction is one of the most important things to understand when choosing an embedding stack.

6.5.1 Stage 1: Ranking (Retrieval)

Ranking is the initial ordering produced by the retriever. In an embedding pipeline:

1. Embed the query
2. Embed all documents or chunks ahead of time
3. Compute similarity
4. Return the top k nearest items

This stage is fast, scalable, and approximate. Document embeddings are precomputed; nearest- neighbor search can be indexed efficiently.

6.5.2 Stage 2: Re-ranking

Re-ranking is a second pass over the candidate set returned by the retriever:

1. Retriever returns top 20, 50, or 100 candidates
2. Reranker reads the query plus each candidate together
3. Reranker assigns a more exact relevance score
4. Candidates are reordered before being passed to generation

This stage is slower and more expensive, but usually more accurate, because the reranker can reason over the direct query-document interaction rather than coarse vector proximity.

6.5.3 Why Not Rerank Everything?

Reranking does not scale to a large corpus. If you have 1M chunks, you can vector-search them efficiently, but you do not want to run a cross-encoder reranker over all 1M. So the pattern is: retrieve broadly, rerank narrowly.

6.5.4 What Each Stage Optimizes For

Stage	Goal
Embedding model	Maximize semantic recall at scale; good-enough ranking
Reranker	Maximize precision at the top of the list

Embedding and reranking scores are not interchangeable. A strong embedding model that gets the right document in the top 20 at high speed is more valuable than a slow but precise model that cannot serve queries under load.

DRAFT

6.5.5 Practical Retrieval Stack

A strong modern local retrieval stack:

1. Embedding model generates query and document vectors
2. ANN index (HNSW) returns top 20–100 candidates (see Chapter 7)
3. Reranker rescoring narrows to best top 5–10
4. Only the highest-ranked chunks go into the final LLM prompt

6.6 Architecture Selection Guide

First local deployment Use dense retrieval with Qwen3-Embedding-4B. Serve with `llama-server --em` Add reranking if top-of-list precision becomes a bottleneck.

Hybrid search needed Add BM25 alongside vector search; fuse results with RRF (see Chapter 8). memvid includes this by default in its `ask()` method.

ColBERT / multi-vector Introduce BGE-M3 if you need multi-aspect query matching and are willing to manage a larger index. Not the first step.

Complex code retrieval ColBERT or sparse retrieval may outperform pure dense for queries with specific identifiers, version numbers, or exact function names.

DRAFT

7

ANN Indexes and Distance Metrics

7.1 The Problem with Exact Search

Given a query vector and a corpus of N document vectors, exact nearest neighbor search computes the distance between the query and every document, then sorts. This is $O(N \times d)$ where d is the dimensionality. At 1M documents with 1536-dim vectors, that is ~1.5B float multiplications per query—too slow for interactive use.

ANN (Approximate Nearest Neighbor) trades a small amount of recall for a large speedup. Instead of finding the provably closest k vectors, it finds a very-probably-close k with much less computation.

7.2 HNSW — Hierarchical Navigable Small World

The dominant ANN algorithm in production vector stores (Qdrant, pgvector, memvid, Weaviate).

The core idea: build a multi-layer graph where each node is a document vector. Upper layers are sparse long-range connections (like highways); lower layers are dense local connections (like local roads). At query time, navigate top-down from the sparse layer to the dense layer, greedily following edges toward the query vector.

```
Layer 2 (sparse):  A            E
Layer 1 (medium):  A    B    D    E
Layer 0 (dense):   A   B   C   D   E   F   G
                                ↑
                            query lands here
```

Key parameters:

Parameter	What it controls	Typical va
`M`	Max edges per node per layer. Higher = better recall, larger index	16
`ef_construction`	Candidate pool at build time. Higher = better graph quality	200
`ef` (search)	Candidate pool at query time. Higher = better recall, slower	50–200

DRAFT

memvid hardcodes M=16, ef_construction=200 with no runtime override. Qdrant and pgvector expose these as configurable at index creation time.

Recall vs. speed: HNSW at ef=50 typically achieves 95–99% recall@10 (meaning 95–99% of the true top-10 nearest neighbors are in the returned set) at 10–100× the speed of brute-force scan. Raising ef improves recall at the cost of latency.

Memory requirement: HNSW graphs are in-memory structures. At 1536 dims × float32 × 1M vectors = ~6 GB just for raw vectors, plus graph overhead. This is the primary scaling ceiling for memvid and pgvector—they are RAM-bound. Qdrant and LanceDB use disk-based HNSW with memory-mapped pages.

7.3 Flat Scan (Brute Force)

At small scale (< ~10k vectors), linear scan with SIMD-accelerated distance computation is often faster than HNSW because there is no graph traversal overhead. memvid uses flat scan below 1,000 vectors. Most vector DBs also fall back to exact scan at small collection sizes.

7.4 IVFFlat (Inverted File)

An older algorithm used in Faiss and pgvector. Clusters vectors into `nlist` Voronoi cells during build. At query time, only searches the `nprobe` nearest cells. Faster to build than HNSW, less memory, but lower recall at the same speed.

pgvector supports both IVFFlat and HNSW. HNSW is generally preferred for recall quality; IVFFlat is useful when build time or memory is the constraint.

7.5 Product Quantization (PQ)

PQ compresses vectors to reduce memory. The vector space is split into M subspaces; each subspace is quantized to one of k centroids. A 1536-dim float32 vector (6,144 bytes) can be compressed to M bytes (one centroid ID per subspace).

memvid's PQ96 splits into 96 subspaces × 256 centroids of 4 dims each → 96 bytes per vector (64× compression of the quantized portion). Search uses Asymmetric Distance Computation (ADC) with lookup tables—the query is not quantized, only the stored vectors are.

Trade-off: PQ degrades recall compared to exact vectors. The more aggressive the compression, the larger the recall loss. PQ is typically combined with HNSW: HNSW navigates the graph, distances are computed via ADC.

Qdrant's scalar quantization (SQ8) is a simpler alternative—quantize each dimension from float32 to int8 (4× compression, lower recall loss than PQ).

7.6 Distance Metrics

7.6.1 Cosine Similarity

Measures the angle between vectors regardless of magnitude:

DRAFT

$$\text{cosine}(a, b) = \frac{a \cdot b}{\|a\| \cdot \|b\|}$$

Values range from -1 (opposite) to 1 (identical direction). The standard metric for most embedding models. Equivalent to dot product on unit-normalized vectors.

7.6.2 L2 Distance (Euclidean Distance)

$$L2(a, b) = \sqrt{\sum_i (a_i - b_i)^2}$$

Smaller L2 distance means the vectors are more similar. Some backends convert L2 to a similarity score via similarity $= 1.0 -$ distance. This only makes sense when vectors are L2-normalized (unit length), because on the unit hypersphere:

$$L2(a, b)^2 = 2 \times (1 - \text{cosine_similarity}(a, b))$$

If vectors are not normalized, $1.0 - L2_\text{distance}$ produces incorrect similarity scores—values can go negative and the scale is arbitrary. Callers must normalize embeddings before storing them if they want cosine-equivalent ranking from L2 search.

memvid hardcodes L2 distance. Callers must normalize externally for cosine equivalence.

7.6.3 Dot Product

$$\text{dot}(a, b) = \sum_i a_i \cdot b_i$$

Equivalent to cosine similarity when vectors are unit-normalized. Some models are trained to maximize dot product rather than cosine similarity and should not be L2-normalized before storage. Check the model card.

7.7 Elbow Cutoff (Score Cliff Detection)

When an ANN search returns a ranked list, not all results are useful. The score distribution typically looks like:

```
rank:  1     2     3     4     5     6     7     8
score: 0.91  0.88  0.86  0.85  0.51  0.49  0.47  0.44
```

There is a sharp drop between rank 4 and rank 5. Everything from rank 5 onward is a different cluster of relevance—probably noise or tangentially related documents.

The **elbow** (or cliff) is that drop point. Rather than returning a fixed `top_k`, adaptive retrieval detects where the score curve bends sharply and truncates there.

Common cutoff strategies (as implemented in memvid's `AdaptiveConfig`):

`AbsoluteThreshold` Drop everything below a fixed score (e.g., 0.6)

`RelativeThreshold` Drop everything below `ratio` × top score

DRAFT

`ScoreCliff` Stop when the score drops more than X% from the previous result

`Elbow` Automatic knee-detection in the score curve (derivative-based)

`Combined` Run multiple strategies; first one to trigger wins

DRAFT

8

Ranking, BM25, and RRF

8.1 BM25 — Best Match 25

BM25 is the dominant full-text ranking algorithm used in search engines (Elasticsearch, Lucene, Solr, Tantivy). It scores documents against a query based on term frequency and inverse document frequency with saturation corrections:

$$\text{score}(D, Q) = \sum_{t \in Q} \text{IDF}(t) \cdot \frac{\text{tf}(t, D) \cdot (k_1 + 1)}{\text{tf}(t, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

Where:

- $\text{tf}(t, D)$ — how many times term t appears in document D
- $\text{IDF}(t)$ — how rare the term is across all documents (log-scaled)
- k_1 — term frequency saturation (typically 1.2–2.0); prevents a term appearing 100× from dominating
- b — length normalization (typically 0.75); penalizes long documents
- avgdl — average document length in the corpus

Why BM25 matters for RAG: Vector search finds semantically similar content but can miss exact keyword matches: names, version numbers, code identifiers, rare technical terms. BM25 excels at exact match and rare term recall. Hybrid search combines both.

Implementation notes: memvid uses Tantivy's BM25. MongoDB Atlas Search uses Lucene's. Qdrant uses FastEmbed's sparse encoding which approximates BM25.

8.2 RRF — Reciprocal Rank Fusion

When you run both BM25 and vector search on the same query, you get two ranked lists. The lists contain overlapping but not identical results with incomparable scores (BM25 scores are not in the same range as cosine similarity values).

RRF merges them by rank position, not by raw score:

$$\text{RRF_score}(d) = \sum_{L} \frac{1}{k + \text{rank}_L(d)}$$

DRAFT

Where k is a constant (typically 60). The k value dampens the advantage of rank 1 over rank 2—it prevents a single list from dominating just because one result was ranked first.

Example with $k = 60$ and two lists:

```
doc A: rank 1 in BM25, rank 4 in vector  → 1/61 + 1/64 = 0.0321
doc B: rank 2 in BM25, rank 2 in vector  → 1/62 + 1/62 = 0.0323
doc C: rank 1 in vector, not in BM25     → 0    + 1/61 = 0.0164
```

Doc B wins because consistent top performance across both lists beats a single dominant rank. Doc C is penalized for appearing in only one list.

RRF requires no calibration of score scales across different retrieval systems—only ranks matter. This makes it robust for combining any retrieval methods without normalization.

memvid's `ask()` method uses RRF with $k = 60$ as the default fusion strategy.

8.3 nDCG in Practice: Using Rankings to Make Better Decisions

Understanding nDCG at a practical level helps you make better decisions about your retrieval stack. The key insight is that nDCG measures ranking quality, not just membership.

8.3.1 Why nDCG Is Better Than Plain Recall for Ranking Quality

Recall asks: did the system retrieve relevant items? It does not strongly care whether the best item was rank 1 or rank 10. nDCG does care. This is why:

- Recall is often more important for stage-1 candidate generation
- nDCG is often more important for top-of-list quality and reranking

8.3.2 Why `@10` Is Common

`nDCG@10` is common because many applications only care about the first screenful of results or the first prompt window. A RAG pipeline may pass only the top 5 to top 10 chunks into generation. If your system only ever uses the top 5 chunks, then `nDCG@5` may be even more informative for your internal evals.

8.3.3 nDCG with Binary vs Graded Judgments

With binary judgments, nDCG mostly reflects whether relevant items are ranked early. With graded judgments, nDCG becomes more expressive—it can reward systems that rank the very best documents above merely acceptable ones. This makes graded nDCG especially valuable for reranking benchmarks.

DRAFT

8.3.4 What nDCG Does Not Tell You

nDCG does not directly tell you:

- Whether your retriever found all relevant items in the corpus
- How expensive the retrieval system is
- Whether the retrieved text is the best chunking strategy for generation
- Whether your model generalizes outside the benchmark

That is why nDCG should usually be read together with Recall@k and domain-specific benchmark slices or internal evaluation.

8.4 SPO and Entity Graphs

SPO triplets are a way to represent facts as structured relationships:

```
Subject        Predicate       Object

Alice          works_at        Acme Corp
Bob            lives_in        San Francisco
project-X      depends_on      library-Y
```

SPO extraction parses free text and produces a graph of entities and their relationships. In retrieval systems, this enables queries that pure vector search cannot handle well:

- "who works at Acme Corp" — entity lookup, not semantic similarity
- "what does project-X depend on" — graph traversal
- "find all relationships involving Alice" — entity-centric search

MemoryCards (as implemented in memvid) are typed records built on top of SPO:

Fact A general assertion ("Paris is the capital of France")

Preference A user preference ("user prefers dark mode")

Event A timestamped occurrence ("meeting with Alice on 2026-03-15")

Relationship A link between entities ("Alice reports to Bob")

In agent memory systems, the SPO graph complements vector search: vector search finds semantically similar content; graph search finds structured facts about known entities.

DRAFT

8.5 Choosing the Right Retrieval Mix

Query type	Best retrieval method
Semantic / conceptual	Dense vector search
Exact keyword (names, IDs, version numbers)	BM25
Mixed / general	Hybrid (BM25 + vector with RRF)
Multi-aspect complex queries	ColBERT or reranker
Structured entity queries	SPO graph search

In practice, running hybrid retrieval (BM25 + vector) as the default and adding a reranker for final ordering captures the vast majority of production retrieval improvements. SPO graph search is valuable for agent memory and knowledge graph use cases but requires additional infrastructure.

DRAFT

Part IV

Building the Pipeline

DRAFT

DRAFT

9

Chunking Strategies

Chunking is the process of splitting documents into smaller pieces before embedding and indexing. It is one of the highest-leverage decisions in a RAG system—more so than model choice in many cases. Poor chunking degrades retrieval quality regardless of how good the embedding model is.

9.1 Why Chunk at All?

Two reasons force chunking:

1. Token limits. Embedding models have a maximum input length (typically 512–8,192 tokens). A document longer than that limit must be split before it can be embedded. Truncation is the naive alternative—it silently discards everything past the limit.

2. Retrieval precision. A vector representing a 20,000-word document compresses too much information into one point. When you retrieve that document in response to a specific question, most of its content is irrelevant to the query. The retrieved context sent to the LLM is bloated with noise.

Chunking trades recall (a whole document contains the answer somewhere) for precision (the specific chunk most relevant to the query is returned).

When Not to Chunk: Long-context generative models (Gemini 2.0, Claude with 200K context) can ingest entire documents. If the corpus is small enough that all documents fit in the model's context window, chunking adds complexity without benefit. Chunking makes sense when: the corpus is too large to fit in one prompt, retrieval must select relevant content from many documents, or the LLM context budget is constrained.

9.2 The Chunk Size Tradeoff

Chunk size is the central tuning parameter:

DRAFT

Smaller chunks	Larger chunks
Higher retrieval precision	Higher recall
Less context per retrieved chunk	More context per chunk
More chunks to index and store	Fewer chunks, smaller index
Better for fact lookup, Q&A	Better for synthesis, summarization
Risk: answer split across boundary	Risk: irrelevant content dilutes chunk

Typical defaults by use case:

Use case	Chunk size
Q&A over technical docs	256–512 tokens
General knowledge retrieval	512–1,024 tokens
Long-form synthesis	1,024–2,048 tokens
Code retrieval	Function/class boundary (variable)

There is no universal optimal. Test on representative queries from your domain.

9.3 Strategy 1: Fixed-Size Chunking

Split every N characters (or tokens), regardless of content boundaries.

```
document = "The model uses attention mechanisms. Attention allows..."

chunk 1 = "The model uses attention mec"   ← breaks mid-word
chunk 2 = "hanisms. Attention allows..."
```

Advantage: Simple. Predictable chunk sizes. Easy to implement.

Disadvantage: Breaks at arbitrary points—mid-sentence, mid-word, mid-concept. Fixed-size chunking without overlap is rarely useful in practice.

9.4 Strategy 2: Fixed-Size with Overlap

Like fixed-size, but each chunk shares some content with the previous and next chunk.

```
chunk_size = 500 tokens
overlap    = 100 tokens

chunk 1: tokens 0-499
chunk 2: tokens 400-899    ← shares 100 tokens with chunk 1
chunk 3: tokens 800-1299   ← shares 100 tokens with chunk 2
```

DRAFT

Why overlap exists: If the relevant answer spans a chunk boundary, overlap ensures the answer appears whole in at least one chunk. Without overlap, a sentence split at the boundary would be partially in chunk N and partially in chunk N+1, and neither chunk would retrieve well.

Disadvantage: Increases index size proportional to overlap fraction. Same content appears multiple times in search results, requiring deduplication.

Typical overlap: 10–20% of chunk size. Higher overlap for fine-grained Q&A; lower for storage-constrained deployments.

9.5 Strategy 3: Sentence-Boundary Chunking

Accumulate text until reaching the size target, but only split at sentence boundaries (‘.’, ‘¡, ‘¿, paragraph breaks).

Advantage: Each chunk is semantically coherent. No broken sentences. Better embedding quality because the model receives grammatically complete input.

Disadvantage: Chunk sizes vary. A single very long sentence can exceed the target. Requires a sentence tokenizer (spaCy, NLTK, or heuristic regex).

This is the preferred baseline for most RAG systems. memvid approximates this with its “natural breakpoint” look-ahead (finding sentence/paragraph endings near the target boundary).

9.6 Strategy 4: Recursive Character Splitting

Split on the most natural boundary available, falling back to coarser splits only when necessary.

```
Priority order: paragraph → sentence → word → character

1. Try splitting on "\n\n" (paragraph breaks)
2. If any resulting chunk is too large, split on ". " (sentences)
3. If still too large, split on " " (words)
4. If still too large, split on "" (characters)
```

This is the default strategy in LangChain and LlamaIndex. It preserves document structure when possible and only resorts to arbitrary splits when forced.

Advantage: Produces coherent chunks across a wide range of document types without per-format configuration.

Disadvantage: Can still break logical units (a numbered list, a table row, a code block) when the target size forces it.

9.7 Strategy 5: Structure-Aware Chunking

Use knowledge of the document’s format to split on meaningful boundaries:

- **Markdown**: split on headers (`##`, `###`), preserve fenced code blocks and tables whole
- **HTML**: split on `<section>`, `<article>`, `<h2>` tags

DRAFT

- **PDF with structure**: split on section headers from the PDF outline
- **Code**: split on function/class definitions, not arbitrary line counts
- **Tables**: keep each row (or the whole table) as a unit; propagate headers to every chunk

Advantage: Chunks align with the document's own logical structure.

Disadvantage: Requires a parser per document type. Section-level chunks can be very long or very short depending on the document.

memvid activates structure-aware mode when it detects Markdown tables or code fences, propagating table headers to continuation chunks.

9.8 Strategy 6: Semantic Chunking

Split based on **topic shift** rather than size targets. Embed each sentence, compute cosine similarity between adjacent sentence vectors, and insert a chunk boundary where similarity drops sharply (a semantic "cliff").

```
sentence 1 → vector
sentence 2 → vector  similarity(1,2) = 0.92  → no split
sentence 3 → vector  similarity(2,3) = 0.89  → no split
sentence 4 → vector  similarity(3,4) = 0.41  → SPLIT HERE
sentence 5 → vector  similarity(4,5) = 0.88  → no split
```

Advantage: Chunk boundaries align with actual topic transitions. Each chunk has a single coherent topic. Retrieval precision improves.

Disadvantage: Requires running the embedding model during indexing (not just at query time). Chunk sizes are unpredictable. Topic shifts within a long sentence are not detected.

Semantic chunking is most valuable for long, multi-topic documents (books, reports, wikis). For short uniform documents (product descriptions, Q&A pairs) it adds cost without benefit.

9.9 Strategy 7: Parent-Child (Small-to-Big) Chunking

Index small chunks for retrieval precision, but return the larger parent chunk to the LLM for context richness.

```
Parent chunk:  full section (1200 tokens)  ← not directly indexed
Child chunks:  each paragraph (~200 tokens) ← these are embedded and indexed

Query → retrieve child chunks → return their parent to the LLM
```

Why this works: Small child chunks match queries precisely. But the LLM benefits from the broader context of the parent section, which includes surrounding sentences that explain the child chunk.

Variants:

- Sentence → paragraph (retrieve sentences, return paragraphs)

DRAFT

- Paragraph → section (retrieve paragraphs, return sections)
- Chunk → full document (retrieve chunks, return whole document)

memvid implements this: `FrameRole::DocumentChunk` for child chunks, `FrameRole::Document` for the parent, linked by `parent_id`.

Disadvantage: Doubles the storage (parent and child both stored). The parent context may still include irrelevant content. Works best when document sections have natural boundaries.

9.10 Strategy 8: Late Chunking

An emerging strategy for ColBERT-style models. Rather than chunking before embedding, embed the whole document and then extract chunk-level embeddings from the token-level outputs.

```
Document → [embedding model] → token vectors (one per token)
                              → pool each chunk's token range into one vector
```

Advantage: Eliminates boundary artifacts—a sentence at the start of the document has the same embedding quality as one in the middle.

Disadvantage: Requires a model that exposes token-level outputs (ColBERT-style or raw transformer outputs). Not supported by the standard OpenAI-compatible embeddings API. Currently experimental.

9.11 How memvid Handles Chunking

memvid's chunking implementation (`src/memvid/chunks.rs`):

- Default: 1,200-character chunks with ±240-char look-ahead for natural breakpoints
- Documents under 2,400 chars are not chunked
- Structure-aware mode activates on detection of Markdown tables or code fences
- Tables: splits between rows, propagates headers to continuation chunks
- Code: keeps fenced blocks whole where possible
- Parent-child: `FrameRole::Document` links to `FrameRole::DocumentChunk` children via `parent_id`; `TextChunkManifest` records each child's `(start, end)` character range

Gaps: no overlap, no semantic chunking, no sentence-boundary splitting (heuristic only), no recursive character fallback. These are reasonable defaults for a general-purpose embedded store but may need supplementing for domain-specific corpora.

DRAFT

9.12 Practical Checklist

When designing a chunking strategy:

1. **Know your documents.** Short uniform records behave differently from long heterogeneous documents. Sample 20–30 representative documents before choosing a strategy.
2. **Know your queries.** Fact-lookup queries benefit from small precise chunks. Synthesis queries benefit from larger chunks.
3. **Start simple.** Sentence-boundary chunks with 15% overlap is a good default. Measure retrieval quality before adding complexity.
4. **Measure chunk size distribution.** Check the p50, p95, and max chunk sizes. Outliers (single-sentence chunks, oversized sections) degrade retrieval differently than average chunks.
5. **Always store metadata.** At minimum: source, section, date. This is cheap to add and expensive to reconstruct later.
6. **Plan for re-chunking.** When you change chunking strategy or embedding model, the entire index must be rebuilt. Design ingestion pipelines so re-chunking is not a one-time manual operation.

DRAFT

10

The RAG Pipeline

Retrieval-Augmented Generation (RAG) is a pattern for answering questions against a corpus too large to fit in a single LLM prompt. A retrieval system selects the relevant subset; the LLM uses that subset to generate a grounded answer.

10.1 Two Phases

RAG has two distinct phases that run at different times and at different frequencies:

```
INDEXING PHASE (offline, once per document)
  Raw document → extract text → chunk → embed → store in index

QUERY PHASE (online, once per question)
  Question → embed → search index → retrieve chunks → assemble context → generate
```

The indexing phase is batch work: slow, expensive, done in advance. The query phase must be fast: the user is waiting.

10.2 Indexing Phase

10.2.1 Step 1: Document Ingestion

Accept the raw document in its source format. Common formats: plain text (`.txt`, `.md`), PDF, HTML, DOCX, XLSX, code.

At this step, also extract metadata: source URL or path, title, author, creation date, document type. This metadata travels with every chunk the document produces.

Failure modes:

- PDF with scanned images: text extraction returns nothing. Requires OCR.
- PDF with ligature fonts: some extractors corrupt *fi*, *fl* ligatures.
- HTML with JavaScript-rendered content: static extraction misses dynamically loaded content.

DRAFT

10.2.2 Step 2: Text Cleaning

Before chunking, normalize the extracted text:

- Remove boilerplate (headers, footers, page numbers, navigation)
- Normalize whitespace and Unicode (ligatures, smart quotes, zero-width spaces)
- Optionally repair broken words from PDF column extraction

Cleaning quality directly affects embedding quality. Embedding a chunk full of OCR artifacts produces a vector that does not cluster with clean content.

10.2.3 Step 3: Chunking

Split the cleaned text into chunks sized for the embedding model's context window and the retrieval precision you need. See Chapter 9 for the full decision tree.

Key decisions at this step:

- Chunk size (characters or tokens)
- Overlap (0–20% is typical)
- Split strategy (fixed, sentence-boundary, structure-aware, semantic)
- Parent-child relationship (store parent for context, index children for retrieval)

Output: a list of `(chunk_text, chunk_metadata)` pairs.

10.2.4 Step 4: Embedding

Pass each chunk through the embedding model to produce a vector:

```
chunk_text  → [embedding model] → [0.21, -0.45, 0.03, ...]
```

This is the most computationally expensive step. Strategies:

- **Batch embedding:** send multiple chunks per API call (up to 2,048 for OpenAI). Reduces round-trip overhead by orders of magnitude vs. one call per chunk.
- **Local vs. API:** local ONNX or GGUF models have zero per-call cost but require GPU/CPU capacity. OpenAI API has per-token cost but no infrastructure overhead.
- **Caching:** if re-indexing a partially updated corpus, skip chunks whose text has not changed (hash the chunk text to detect unchanged content).

DRAFT

10.2.5 Step 5: Storing in the Index

Write each `(vector, chunk_text, chunk_metadata)` into the storage backend.

What gets stored:

- The vector (for ANN search)
- The chunk text (to return as context to the LLM)
- The metadata (for filtering and attribution)
- Optionally: a BM25 inverted index over the chunk text (for hybrid search)

10.2.6 Indexing Pipeline Summary

```
[raw docs]


[extract text]  ←  format-specific parsers


[clean text]    ←  normalize, strip boilerplate


[chunk]         ←  size, overlap, strategy choice


[embed]         ←  model choice, batch size, caching


[store]         ←  vector index + optional BM25 index
```

10.3 Query Phase

10.3.1 Step 1: Receive the Query

The user submits a question or search string. Before retrieval, consider:

- **Query type:** Is this a factual lookup, a synthesis request, or a conversational follow-up?
- **Query transformation:** Should the query be rewritten before retrieval?

Common query transformation techniques:

HyDE (Hypothetical Document Embedding) Ask the LLM to write a hypothetical document that would answer the question, then embed that document instead of the bare query. Bridges the vocabulary gap between short queries and long documents.

Query expansion Append related terms to improve recall.

Sub-question decomposition Split a complex query into simpler sub-questions, retrieve for each, then combine.

DRAFT

10.3.2 Step 2: Embed the Query

Embed the query using the same model used to embed the documents. This is non-negotiable—the query and document vectors must live in the same vector space.

```
query_text → [same embedding model] → query_vector
```

If query instructions are used (as with Qwen3-Embedding), wrap the query in the appropriate instruction template before embedding. See Chapter 13.

10.3.3 Step 3: Retrieve Candidates

Pure vector search:

```
query_vector → ANN search → top-k chunks ranked by vector similarity
```

Hybrid search (BM25 + vector):

```
query_text   → BM25 search → lex_results (ranked by keyword relevance)
query_vector → ANN search  → vec_results (ranked by semantic similarity)
lex_results + vec_results   → RRF fusion → merged_results
```

Hybrid search consistently outperforms either modality alone, especially on queries with exact-match terms or where the vocabulary gap between query and document is small.

10.3.4 Step 4: Filter

Apply metadata filters to the candidate set:

- Date range ("only documents from the last 6 months")
- Source type ("only from the API documentation")
- Access control ("only documents this user can see")

Filtering is most efficient when done **during** ANN search (as supported by Qdrant and MongoDB Atlas) rather than after (as in memvid). Post-filtering on a small top-k can silently produce fewer results than requested if many candidates are filtered out.

10.3.5 Step 5: Rerank (Optional but Recommended)

Pass the top candidates through a cross-encoder reranker to produce a more precise ordering.

```
(query, candidate_1) → [reranker] → 0.92
(query, candidate_2) → [reranker] → 0.87
(query, candidate_3) → [reranker] → 0.41
```

The reranker reads the query and each candidate together, producing a relevance score that accounts for the full text interaction. Reranking is skipped when: latency is critical, the corpus is small enough that vector search precision is already high, or no reranker model is available.

DRAFT

10.3.6 Step 6: Assemble Context

Select the top N chunks and concatenate them into a context block for the LLM prompt.

Context budget: The LLM has a finite context window. A practical rule: reserve 30% for the answer, 10% for system prompt and question, 60% for retrieved context.

Ordering matters: LLMs attend less reliably to content in the middle of long contexts (the "lost in the middle" problem). Put the most relevant chunks first and last.

Attribution: Include source metadata so the LLM can cite sources:

```
[Source: retrieval-concepts.md, Section: HNSW]
HNSW builds a multi-layer graph where each node is a document vector...
```

Parent-child expansion: If using parent-child chunking, swap retrieved child chunks for their parent chunk at this step to give the LLM richer context.

10.3.7 Step 7: Generate the Answer

Typical prompt structure:

```
SYSTEM:
You are a helpful assistant. Answer using only the provided context.
If the context does not contain the answer, say so.

CONTEXT:
[assembled context block]

USER:
[original question]

ASSISTANT:
```

The system prompt should instruct the LLM to use the provided context as the primary source, acknowledge when the context does not contain the answer, and cite sources when relevant.

10.4 Common Failure Modes

10.4.1 Retrieval Failures

Symptom: The LLM says "I don't know" but the answer is in the corpus.

Probable causes:

- Chunk boundary split the answer across two chunks
- Query vocabulary doesn't match document vocabulary (semantic gap); try hybrid search or HyDE
- `top_k` too small; the right chunk is at rank 12 but only top 5 are retrieved

DRAFT

- Metadata filter too aggressive; filtered out the relevant chunk

Symptom: Retrieved chunks are consistently off-topic.
Probable causes:

- Chunks are too large; the relevant sentence is buried in an irrelevant chunk
- Embedding model is not well-suited to the domain
- No reranker; vector proximity is imprecise for this query type

10.4.2 Generation Failures

Symptom: LLM ignores the context and answers from its parametric memory.
Probable causes:

- System prompt does not strongly instruct the model to use context
- Context block is too long; relevant chunks are in the "lost in the middle" zone
- Model temperature too high; try 0.0–0.3 for fact retrieval tasks

10.4.3 Indexing Failures

Symptom: Re-indexing takes much longer than expected.
Probable causes:

- Embedding every chunk individually instead of batching
- No caching of embeddings for unchanged content

Symptom: Index quality degrades over time as documents are updated.
Probable causes:

- Stale vectors from old document versions still in the index
- Backend does not support correct deletions (see Chapter 11)
- No mechanism to detect and re-embed changed content

10.5 This Repo's Stack

In this repo, the RAG pipeline maps to the following components:

DRAFT

Pipeline step	Component
Document ingestion	memvid readers (PDF, DOCX, XLSX) or direct `put_bytes()`
Chunking	memvid chunker (1,200 char, structure-aware)
Embedding	`llama-server --embedding` serving a GGUF embedding model
Storage	memvid `.mv2` file (vector + BM25 + metadata)
Query embedding	Same llama-server endpoint
Retrieval	memvid `ask()` — hybrid BM25 + vector with RRF
Reranking	Not yet integrated (planned: Qwen3-Reranker via llama-server)
Context assembly	memvid `AskResponse.context` field
Generation	llama-server chat completions or cloud provider via clawrouter

The embedding model and generation model are served as separate llama-server instances. Open WebUI routes embedding requests to the dedicated embeddings endpoint and generation requests to the generation endpoint. clawrouter handles routing to local vs. cloud providers.

DRAFT

DRAFT

11

Retrieval Backends

This chapter compares storage backends for medium-scale RAG and agent memory use cases. Primary reference: analysis of `/d/memvid` (memvid-core v2.0.139, Rust) conducted April 2026.

11.1 memvid v2 — Deep Analysis

11.1.1 What It Is

A single `.mv2` binary file containing everything: compressed payloads (zstd/lz4), a WAL, a Tantivy BM25 index, an HNSW or flat vector index (bincode-serialized), a time index, a CLIP visual index, an SPO entity-relationship graph, and typed "MemoryCard" facts. No sidecar files. The entire corpus is one portable, optionally-encrypted artifact.

The name is a metaphor—there is no video codec. "Frames" are immutable append-only content units inspired by video frame semantics. The v1 library (deprecated Python) used QR codes embedded in MP4 files; v2 discards that entirely.

11.1.2 Storage and Embedding Mechanics

Embeddings are stored as raw `Vec<f32>` in the WAL at ingest time, materialized into the vector index on `commit()`.

Index selection at build time:

- $<$ **1,000 vectors** $\rightarrow$ flat scan (bincode `Vec<VecDocument>`, linear L2 with SIMD)
- $\geq$ **1,000 vectors** $\rightarrow$ HNSW (M=16, ef_construction=200, hardcoded constants)
- **Optional PQ96** $\rightarrow$ 16$\times$ compression via product quantization, needs $\geq$ 100 vectors

Distance metric is **L2 (Euclidean)** throughout. Similarity score $= 1.0 - \text{distance}$. Callers must normalize vectors externally for cosine equivalence.

Supported embedding sources:

- Local ONNX (BGE-small 384d, BGE-base 768d, nomic-embed-text 768d, gte-large 1024d)
- OpenAI API (text-embedding-3-small 1536d, text-embedding-3-large 3072d, ada-002 1536d)
- CLIP visual (MobileCLIP-S2, 512d)

DRAFT

- Whisper audio (transcription → text → embed)
- Caller-provided `Vec<f32>` via `put_with_embedding()`

11.1.3 Retrieval Patterns

Method	What it does
`search()` / `search_lex()`	BM25 via Tantivy with sketch pre-filter
`search_vec()`	Pure ANN (flat or HNSW), returns distance scores
`search_adaptive()`	Vector with score cutoff strategies (cliff, elbow)
`ask()`	Hybrid BM25 + vector with RRF (k=60), intent detection
`search_clip()`	Visual similarity over CLIP embeddings
`graph_search()`	Entity-relationship pattern matching over SPO MemoryCards
`timeline()`	Chronological retrieval with time-travel support

`ask()` adjusts `top_k` multipliers by intent: 1× (default), 2× (recency), 3× (aggregation), 5× (analytical).

11.1.4 Memory and Agent Features

`MemoryCard` types: facts, preferences, events, relationships. Populated via:

- Explicit `put_memory_card()` API call
- Auto-extraction by enrichment workers (background, feature-gated)
- SPO triplet extractor that parses Subject-Predicate-Object from content

`Logic-Mesh` (entity graph) enables `graph_search()` for natural language entity queries ("who works at", "who likes", "lives in").

11.1.5 Performance Profile

From their own benchmarks on local retrieval:

- P50: **0.025ms**
- P99: **0.075ms**

DRAFT

11.1.6 Hard Limitations

Limitation	Impact
HNSW deletions are no-ops	Deleted frames remain in vector index
Cannot retrieve original embedding from HNSW/PQ	No export without full rebuild
PQ96 and HNSW params hardcoded	No recall/speed tuning
No pre-filtering during ANN	Metadata filters applied after retrieval only
Fully synchronous API	Requires `spawn_blocking` in async Rust
Temporal TZ hardcoded to `America/Chicago`	Incorrect timestamps for non-US deployments
No distributed mode	Single-machine ceiling

The HNSW deletion no-op is the most significant production concern. Any corpus needing forgetting/eviction semantics must call `finalize_indexes()` (full-scan rebuild) to actually purge deleted embeddings.

11.2 Backend Comparison Matrix

	memvid	Qdrant	LanceDB	pgvector	Chro
Deployment	`.mv2` file	Docker	Embedded	PG extension	Server
Self-hosted	Trivial	Easy	Trivial	Easy	Easy
Hybrid search	BM25+vec+RRF	Sparse+dense	vec only	Manual FTS+vec	vec on
Pre-filter ANN	No	Yes	Yes	Approx	Post o
Deletion	HNSW: no-op	Yes	Yes (versioned)	Yes	Yes
Scale ceiling	5–20M	100M+	100M+	50M+	<5M
Agent/memory	MemoryCards, SPO	None	None	None	None

11.3 Key Backend Details

11.3.1 Qdrant

Self-hosted via Docker, Rust-native, disk-based HNSW with correct deletions. Inline payload filtering during ANN search (exact pre-filter at small scale, sampling at large scale). Sparse + dense hybrid search using FastEmbed for on-device BM25. Named collections map cleanly to per-model or per-agent namespaces.

Quantization options are configurable at index creation time (scalar int8, PQ, binary). REST and gRPC APIs. Strong operational tooling (Prometheus, dashboard).

Best fit for: medium-scale RAG with frequent updates, metadata-filtered ANN, self-hosted.

DRAFT

11.3.2 LanceDB

Embedded (zero server), Rust-based, Apache Arrow + Lance columnar format. Supports S3 and local disk. Versioned snapshots give correct delete/update semantics without a separate server process. Python and Rust APIs both async.

Scale ceiling is much higher than memvid (100M+ vectors) with disk-based storage. Does not include BM25 natively—Tantivy integration is an optional add-on.

Best fit for: large-scale RAG, columnar/tabular data, S3-backed or edge deployment, zero ops.

11.3.3 MongoDB Atlas

Vector search requires Atlas cloud or the `mongot` sidecar process. **Not available in community `mongod`.** This is the primary gate for local-first deployments.

When available: HNSW with configurable similarity metrics, Atlas Search (Lucene-based BM25, facets, fuzzy), hybrid retrieval via `$unionWith` + RRF scoring pipeline.

Best fit for: heterogeneous memory shapes, complex query logic, teams already using Atlas, willing to accept cloud dependency or mongot overhead.

11.3.4 pgvector

PostgreSQL extension. Adds `vector` type, cosine operator, IVFFlat and HNSW indexes. Standard SQL for filtering, joining, and ranking.

Best fit for: teams already running Postgres, structured metadata + vector hybrid queries, medium scale (<50M vectors with tuning).

11.4 Decision Guide

Scenario	Pick
Agent long-term memory, time-travel, portable, air-gapped	**memvid**
Medium-scale RAG, updates/deletes, metadata-filtered ANN	**Qdrant**
Large-scale RAG, columnar, S3-backed, zero-server	**LanceDB**
Heterogeneous memory shapes, complex queries, Atlas acceptable	**MongoDB Atlas**
Existing Postgres infrastructure	**pgvector**
Prototype or small-scale Python RAG	**Chroma**
Ultra-lightweight single-file + SQL	**sqlite-vec**

11.4.1 Stacking memvid with Another Backend

memvid's agent memory features (MemoryCards, SPO graph, timeline, time-travel) are unique and not easily replicated. Its RAG limitations (HNSW deletion no-op, no pre-filtering) are real.

A viable hybrid: use **memvid for agent memory** (structured facts, conversation history, time-travel replay) and **Qdrant for document RAG** (large corpus, filtered ANN, correct

DRAFT

deletes). Route queries through clawrouter with memory context injected from memvid alongside document context from Qdrant.

DRAFT

DRAFT

Part V

Local Deployment

DRAFT

DRAFT

12

llama-server and llama-embedding

12.1 What llama-embedding Does

`llama-embedding` is the llama.cpp command-line tool for generating embeddings directly from a model instead of generating text. Conceptually it:

1. Loads an embedding-capable model
2. Tokenizes the input text
3. Runs a forward pass through the model
4. Extracts a vector representation from the model output
5. Optionally normalizes or formats that vector for downstream use

Instead of returning next-token probabilities or generated text, it returns a numeric vector that represents the semantic meaning of the input.

12.2 What Embedding Generation Means at the Model Level

A transformer processes input tokens into hidden states. An embedding model then turns those hidden states into a single fixed-size vector. That vector is what you store in a vector index and compare during retrieval.

The key challenge is that token sequences have variable length but vector search wants a fixed-size output. So the tool needs a pooling strategy.

12.3 Why Pooling Matters

Pooling is the method used to collapse token-level hidden states into one embedding vector. Common pooling strategies:

`last` The final token representation (used by Qwen3 embedding models)

`mean` Average of all token representations

`cls` The special `[CLS]` token representation (BERT-style)

DRAFT

For Qwen3 embedding GGUF models, the official card explicitly requires:

```
--pooling last
```

This matters because using the wrong pooling mode can degrade retrieval quality even if the model "runs." For Qwen3 embedding models, `last` is not just an implementation detail—it is part of using the model correctly.

12.4 llama-embedding vs llama-server `--embedding`

These serve different roles.

`llama-embedding`:

- One-shot CLI tool
- Useful for local experiments
- Useful for benchmarking
- Useful for generating vectors offline in scripts or batch jobs

`llama-server --embedding`:

- Long-running service
- Exposes an HTTP endpoint (`/v1/embeddings`)
- Better for OpenAI-compatible clients and apps like Open WebUI
- Better when embeddings need to be requested repeatedly

The rough split is: use `llama-embedding` to inspect and test; use `llama-server --embedding` to serve a real application.

12.5 Why Embedding Models Are Often Served Separately

A chat model server and an embedding model server usually have different jobs:

- Chat generation wants `/v1/chat/completions`
- Embedding search wants `/v1/embeddings`

They may also need different models, different ports, different throughput assumptions, and different memory budgets. That is why a dedicated embedding server is usually cleaner than trying to make one chat model do everything.

DRAFT

12.6 What Gets Indexed

In a retrieval workflow, you usually do not embed the entire corpus at query time. Instead:

1. Split documents into chunks
2. Run `llama-embedding` or an embedding server over those chunks once
3. Store the vectors in a vector DB or ANN index
4. At query time, embed only the query
5. Compare query vector to stored document vectors

This is what makes embedding retrieval scalable.

12.7 Why Normalization Matters

Vector similarity is often computed with cosine similarity, dot product, or inner product. Many retrieval systems either explicitly normalize vectors or assume a similarity convention that effectively depends on normalization behavior.

The practical takeaway: keep the indexing and query pipelines consistent. Do not change vector post-processing halfway through a deployment. If query vectors and document vectors are produced differently, retrieval quality can silently degrade.

12.8 Why Embedding Performance Differs from Generation Performance

Embedding workloads are more like: encode text, return one vector. Generation workloads are: repeatedly decode tokens step by step.

So a model that feels fine for embedding may be much too slow for chat generation, and vice versa. That is another reason embedding-specialized models are valuable.

12.9 Practical llama-server Invocations

Start an embedding server with Qwen3-Embedding-4B-GGUF:

```
llama-server \
  --model artifacts/Qwen3-Embedding-4B-Q4_K_M.gguf \
  --embedding \
  --pooling last \
  --port 8081 \
  --ctx-size 4096
```

Verify the endpoint is live:

DRAFT

```
curl -s http://localhost:8081/v1/embeddings \
  -H "Content-Type: application/json" \
  -d '{"input": "test query", "model": "qwen3-embedding"}' \
  | python3 -m json.tool | head -20
```

Generate a single embedding from the CLI:

```
llama-embedding \
  --model artifacts/Qwen3-Embedding-4B-Q4_K_M.gguf \
  --pooling last \
  --prompt "Instruct: Retrieve a relevant passage.
Query: how does KV cache work in llama.cpp?"
```

Run-server.py integration (via `MODEL_SLUG`):

```
MODEL_SLUG=qwen3-embedding-4b ./run-server.py
```

12.10 Practical Deployment Pattern for This Repo

If this repo adopts `Qwen3-Embedding-4B-GGUF`, the most likely operational pattern is:

1. Use `llama-embedding` to validate quality and inspect behavior locally
2. Serve the model with `llama-server --embedding --pooling last`
3. Point Open WebUI or another RAG client at that dedicated embeddings endpoint
4. Later add a Qwen3-Reranker if top-of-list quality becomes the bottleneck

The embedding model and generation model run as separate `llama-server` instances, usually on different ports. This avoids context-switch overhead and allows independent tuning of each model's memory budget.

Why the embedding model should come first: If you only add one model first, the embedding model is the right first step because it gives you the core `/v1/embeddings` capability, is the prerequisite for scalable corpus search, and Open WebUI and RAG systems can immediately benefit from it. Generation quality can be improved later by upgrading the chat model; retrieval quality requires the embedding model to exist at all.

DRAFT

13

Query Instructions and Operational Guidance

DRAFT

13.1 Why Query-Side Instructions Help

The Qwen embedding cards include guidance such as:

> *We recommend that developers customize the instruct according to their specific scenarios, tasks, and languages. Our tests have shown that in most retrieval scenarios, not using an instruct on the query side can lead to a drop in retrieval performance by approximately 1% to 5%.*

Embedding models are not only learning generic "semantic similarity." They are also learning task-conditioned similarity. Those are different questions:

- "what texts are generally similar?"
- "what documents answer this search query?"
- "what code snippet is relevant to this bug report?"
- "what multilingual passage best matches this English question?"

An instruction helps disambiguate the retrieval intent. Without an instruction, the model may optimize for a vaguer notion of semantic closeness. With an instruction, the model can better align the query embedding toward the retrieval objective you actually care about.

13.2 The Basic Pattern

- Query side: add an instruction prefix
- Document side: embed the raw document or chunk (no instruction)

Conceptually:

```
Query embedding input:
"Instruct: Given a web search query, retrieve relevant passages
that answer the query.
Query: how does KV cache reuse work in llama.cpp?"

Document embedding input:
"KV cache reuse in llama.cpp allows ..."
```

The instruction tells the model what "relevant" means for this query.

13.3 Why Not Instruct Both Sides

You usually do not want to add the same instruction to every document chunk because:

- It adds noise and wasted tokens
- Document embeddings are often precomputed once and reused many times
- The task conditioning is usually needed most on the query side

The query is where the ambiguity lives.

13.4 What "Customize the Instruct" Really Means

It does not mean you need a different prompt for every single query. It usually means:

- Choose a stable instruction template for each retrieval use case
- Adapt it to your language and domain
- Keep it consistent during indexing and evaluation

Examples of retrieval modes that may want different instructions:

- Documentation search
- Code search
- Support-ticket retrieval
- Multilingual knowledge-base retrieval
- Legal or medical retrieval with specialized vocabulary

DRAFT

13.5 Example Instruction Templates

General documentation retrieval:

```
Instruct: Given a question, retrieve passages that best answer it.
Query: {query}
```

Code retrieval:

```
Instruct: Given a software engineering question, retrieve code or
documentation relevant to solving it.
Query: {query}
```

Multilingual retrieval:

```
Instruct: Given a multilingual search query, retrieve passages with
the most relevant semantic meaning, even across languages.
Query: {query}
```

Internal knowledge-base retrieval:

```
Instruct: Given an internal operations question, retrieve documents
that directly help answer the question.
Query: {query}
```

13.6 What the Reported 1%–5% Drop Means

That reported drop should be interpreted as: not catastrophic, but large enough to matter in retrieval systems.

Why it matters: a few percentage points in retrieval can be the difference between the right chunk appearing in the top 5 versus disappearing below the cutoff. Once a relevant document misses the first-stage retrieval window, reranking cannot recover it. So even a modest drop at the embedding stage can have a noticeable end-to-end effect on RAG quality.

13.7 Operational Implication

If you adopt a Qwen3 embedding model in this repo, you should plan for:

1. A consistent query instruction template
2. A retrieval-specific evaluation pass using your own corpus
3. Measuring top-k recall before and after changing the instruction

This matters more than chasing leaderboard decimals, because the best instruction for your language mix, chunking strategy, domain vocabulary, and query style may differ from the default examples in the model card.

DRAFT

Practical rule for first deployment: Start with a simple, stable query instruction. Do not instruct documents. Benchmark retrieval quality on your own data. Only then tune the instruction wording. That is the low-complexity path to capturing most of the benefit the Qwen guidance is pointing at.

13.8 Operational Checklist

13.8.1 Before First Deployment

- Confirm the RTEB leaderboard entry for your chosen model (or confirm it is absent and accept the uncertainty)
- Run 50–100 representative query-document pairs through the model and check top-5 recall manually
- Choose a query instruction template appropriate to your domain and record it
- Decide on chunk size, overlap, and strategy before indexing (re-indexing later is expensive)
- Ensure the llama-server port for embeddings is distinct from the chat model port

13.8.2 After Deployment

- Monitor embedding latency separately from generation latency
- Re-embed and re-index when you change the embedding model or chunk strategy
- When adding new document types (code, PDF, HTML), validate that the chunker handles them correctly before bulk ingestion
- If retrieval precision drops after a corpus update, check for stale vectors from deleted or updated documents

13.8.3 Upgrading the Stack

1. **First step:** embed + basic vector search (current target)
2. **Second step:** add a Qwen3 reranker for top-of-list quality
3. **Third step:** add hybrid search (BM25 + vector with RRF) if exact-match queries are failing
4. **Fourth step:** evaluate ColBERT or multi-vector retrieval for complex multi-aspect queries

Each step should be gated on measurement: only add complexity when you have data showing it helps. The first step—a correctly configured embedding server with a good query instruction template—captures the majority of the improvement available.

DRAFT

Glossary

Definitions for terms used throughout this book. Terms in **bold** within a definition are themselves defined in this glossary.

ANN (Approximate Nearest Neighbor) A family of algorithms that find vectors close to a query vector without checking every vector in the index. Trades a small amount of recall for large speedups over exact search. Common algorithms: **HNSW**, **IVFFlat**.

Asymmetric Distance Computation (ADC) A technique used with **Product Quantization** where the query vector is kept in full precision but stored document vectors are quantized. The query is compared against lookup tables of centroid distances.

Bi-encoder A retrieval architecture where query and document are encoded independently by the same or compatible models, producing separate vectors. Fast because document vectors can be precomputed. Contrast with **cross-encoder**.

BM25 (Best Match 25) A full-text ranking algorithm based on term frequency, inverse document frequency, and document length normalization. The standard algorithm behind Elasticsearch, Lucene, Solr, and Tantivy. Excels at exact keyword matching and rare term retrieval.

BPE (Byte Pair Encoding) A **tokenization** algorithm that iteratively merges the most frequent adjacent byte pairs into a single token. Used by GPT-2, GPT-3, GPT-4, LLaMA, Qwen, and most modern LLMs.

Chunking Splitting a document into smaller pieces before embedding and indexing. Necessary because embedding models have token limits and because smaller chunks improve retrieval precision.

ColBERT A retrieval model that keeps one vector per token rather than collapsing to a single vector. Similarity is computed via **MaxSim**: each query token finds its best-matching document token. Higher recall on complex queries; larger index.

Contrastive learning A training objective where the model learns to bring similar examples close together and push dissimilar examples apart in vector space. Uses (anchor, positive, negative) triplets.

Cosine similarity A distance metric measuring the angle between two vectors, regardless of magnitude. Values range from -1 (opposite) to 1 (same direction). The standard metric for most embedding models. Equivalent to **dot product** on unit-normalized vectors.

DRAFT

Cross-encoder A model that reads query and document together as a single input, producing a single relevance score. More accurate than **bi-encoders** because it reasons over query-document interaction. Slower because it cannot precompute. Used as a **reranker**.

Dense retrieval Retrieval using a single continuous vector per text. Contrast with **sparse retrieval** and **ColBERT**.

Embedding A fixed-size vector of floating-point numbers representing the semantic content of a text. Produced by an embedding model. Similar texts produce similar (nearby) vectors.

GGUF A binary file format for quantized model weights, used by llama.cpp and llama-server. Enables running large models on consumer hardware by reducing precision (e.g., Q4_K_M, Q8_0).

Hard negative A training example that appears relevant to the query but is actually not the correct match. Hard negatives force the embedding model to learn fine-grained distinctions.

HNSW (Hierarchical Navigable Small World) The dominant **ANN** algorithm. Builds a multi-layer graph of vectors; upper layers are sparse for long-range navigation, lower layers are dense for local search. Key parameters: M, ef_construction, ef.

HyDE (Hypothetical Document Embedding) A query transformation technique: ask an LLM to write a hypothetical document that would answer the question, then embed that document as the query vector. Bridges the vocabulary gap between short queries and long documents.

Hybrid search Combining multiple retrieval modalities (typically **BM25** and **dense retrieval**) and fusing the results with **RRF** or other merging strategies. Consistently outperforms either modality alone.

L2 distance (Euclidean distance) Straight-line distance between two vectors: $\sqrt{\sum_i (a_i - b_i)^2}$. Smaller = more similar. Equivalent to cosine distance on unit-normalized vectors.

Late chunking Embedding the full document first to preserve context, then extracting chunk-level vectors from the token-level outputs. Avoids boundary artifacts. Requires token-level model outputs.

MemoryCard A typed fact record in memvid's memory system. Types: Fact, Preference, Event, Relationship. Built on top of **SPO** triplets.

MTEB (Massive Text Embedding Benchmark) The primary benchmark suite for evaluating embedding models across multiple task types (retrieval, classification, clustering, reranking, STS). High MTEB scores may reflect benchmark proximity rather than out-of-distribution generalization.

nDCG (Normalized Discounted Cumulative Gain) The primary metric in most retrieval benchmarks. Measures ranking quality: highly relevant documents appearing earlier receive higher scores. Values range from 0 to 1; 1 = perfect ranking. `nDCG@10` = nDCG considering only the top 10 results.

DRAFT

Pooling The step that collapses per-token vectors from a transformer encoder into a single vector. Common strategies: mean pooling, CLS pooling, last-token pooling. The pooling strategy is part of the model contract and must match how the model was trained.

PQ (Product Quantization) A vector compression technique that splits each vector into M subspaces and quantizes each subspace to one of k centroids. Reduces storage by 4–64× at the cost of some recall.

Query instruction A task-specific prefix added to the query before embedding. Improves retrieval quality by 1–5% on models trained with instructions (Qwen3, nomic-embed-text).

RAG (Retrieval-Augmented Generation) A pattern combining retrieval (finding relevant documents) with generation (producing an answer). The LLM generates answers conditioned on retrieved context rather than relying solely on parametric memory.

Recall@k The fraction of all relevant documents that appear in the top-k retrieved results. Contrast with **Precision@k**.

Reranker A **cross-encoder** model used as a second pass over candidates returned by first-stage retrieval. Produces more accurate relevance scores. Applied to a small candidate pool (20–100 items).

RRF (Reciprocal Rank Fusion) A method for merging multiple ranked result lists by position rather than score. Each result scores $\sum 1/(k + \text{rank})$. Requires no score normalization.

RTEB (Retrieval Text Embedding Benchmark) A benchmark designed to test embedding models on retrieval-specific tasks with held-out evaluation datasets not seen during training. Intended to detect benchmark overfitting.

SPLADE A sparse retrieval model that produces learned sparse vectors (one weight per vocabulary term) rather than dense vectors. Complementary to dense retrieval in hybrid search.

SPO (Subject-Predicate-Object) A structured representation of a fact as a triple. Used to build entity graphs in agent memory systems.

Tokenization The process of splitting text into tokens—the basic units a model processes. Common algorithms: **BPE**, WordPiece, SentencePiece.

Transformer The neural network architecture underlying all modern embedding and generation models. Key components: multi-head self-attention, feed-forward layers, positional encoding.

DRAFT